

AI-Linked Equity Crash Stress Model

Methodology, Calibration, Probability Model, and Implementation

Alex Zhang

August 2026

This paper develops a conditional stress model for an AI-linked equity crash. The model separates broad-market de-risking from incremental AI/TMT repricing and size-related fragility, maps those shocks to country-industry and single-stock exposures, and estimates a separate probability distribution across five severity cases conditional on a qualifying bubble burst. It is a model of crash severity and portfolio impact, not a market-timing model.

Abstract

This paper turns an AI-linked equity crash scenario into a transparent cross-sectional stress model. For country or regional bucket b , industry bucket g , market-cap bucket m , scenario s , and horizon h , the stressed log return is

$$R_{b,g,m}^{s,h} = \beta_{b,g}^W S_W^{s,h} + I_{b,g}^{AI} S_X^{s,h} + A_m^{s,h}.$$

The first term captures broad-market de-risking. The second captures incremental AI/TMT repricing after removing ordinary market exposure. The third is an independently calibrated market-cap fragility overlay. The cross-sectional AI vulnerability I^{AI} places **60% weight on statistical evidence and 40% on economic relevance**; statistical evidence is therefore the primary signal, while economic structure stabilizes inference where return histories or proxies are imperfect.

The shock geometry is calibrated from the dot-com unwind. A valuation ratio determines the Base fraction of the full historical TMT-excess channel, with $\kappa_{Base} = 0.670739$. The five-case severity ladder uses $\kappa = 0.335, 0.671, 1.0, 1.5, 2.0$ for Mild, Base, Severe, Extreme, and Worst. Fixed 1-week, 2-week, 1-month, and 2-month shocks use the observed post-peak historical path rather than linear time scaling. Approved single-stock rows apply the same factor equation at ticker level and replace, rather than add to, generic country-industry rows.

The probability layer treats the five deterministic scenarios as representative points of a continuous burst-severity distribution. Using 21 exact U.S. industry crash episodes, the model fits four parsimonious conditional severity families, model-averages them using AICc weights, and quantifies uncertainty with 5,000 episode-level bootstrap refits. The resulting probabilities, conditional on a qualifying burst, are approximately **1.3% Mild, 46.5% Base, 37.6% Severe, 12.5% Extreme, and 2.1% Worst**. They do not estimate the probability that a burst occurs or when it occurs.

The paper gives both the mathematical specification and the implementation. Each major model block is followed by the Python that implements it, and the same source files are included separately in the accompanying bundle. Return estimation, orthogonalization, exposure construction, scenario calibration, overlays, probability assignment, and portfolio mapping can therefore be checked directly against the equations rather than inferred from prose.

Contents

1	Model overview and key results	4
1.1	Key model parameters	4
2	Model objective, scope, and conditioning	5
2.1	Assets and use cases	5
3	Data and calibration roles	5
4	Core model	6
5	Return construction and statistical estimation	7
5.1	Monthly log-return frequency	7
5.2	Broad-market beta	7
5.3	HAC inference and reliability	7
6	Orthogonalizing the AI thematic factor	9
7	Derivation of beta-adjusted statistical AI-excess intensity	10
7.1	Raw co-movement is not incremental exposure	10
8	Economic relevance and the 60/40 blend	11
8.1	Sensitivity of the blend	12

9 Identification as a change of factor basis	12
9.1 Why the model does not use a two-factor regression coefficient directly	13
9.2 Identification assumptions	13
10 Country-industry lift and coherence projection	14
11 Historical analogue selection and the role of dot-com	15
12 Historical shock calibration: broad market versus AI/TMT excess	16
13 Valuation calibration and the Base severity scalar	16
14 Five-case terminal scenario ladder	18
15 Peak-anchored fixed-horizon mechanics	18
15.1 Calibrated fixed-horizon surface	19
16 Stress sensitivities, monotonicity, and nonlinear P&L	20
16.1 Attribution under parameter changes	21
17 Country-industry stress calculation and attribution	21
18 Single-stock override layer	22
19 Market-cap fragility as a separate risk channel	23
19.1 Empirical calibration	23
20 Portfolio application and loss aggregation	24
21 Probability model: objective and statistical object	26
22 Probability geometry: positive log loss and Voronoi cells	26
23 Historical severity sample and empirical identification	27
24 Conditional severity distributions and likelihood construction	28
24.1 Beta distribution on normalized drawdown	28
24.2 Gamma, Weibull, and lognormal on excess log loss	28
25 AICc model averaging	28
26 Bootstrap uncertainty, low-assumption cross-check, and tail diagnostic	29
26.1 Episode-level bootstrap	29
26.2 Jeffreys-prior Dirichlet cross-check	29
26.3 Why generalized Pareto is diagnostic rather than baseline	29
27 Probability model details: estimators, quantization, and uncertainty	30
27.1 Maximum-likelihood parameterization	30

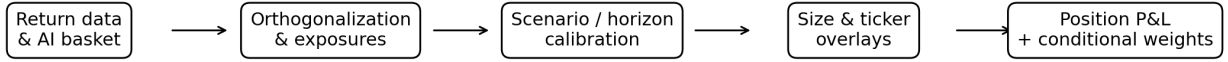
27.2 The five cases as a constrained scenario-reduction problem	30
27.3 Why AICc is used rather than ordinary AIC	30
27.4 Bootstrap uncertainty contains several sources at once	31
27.5 Mean-preservation is a diagnostic, not an estimation target	31
27.6 Probability-weighted portfolio metrics are conditional summaries	31
28 Conditional scenario probabilities	31
29 Model validation	34
29.1 Statistical invariants	34
29.2 Exposure invariants	34
29.3 Scenario invariants	35
29.4 Overlay and mapping invariants	35
29.5 Probability invariants	35
30 Implementation and reproducibility	36
31 Model limitations and controls	36
31.1 Historical analogue risk	36
31.2 Proxy and basis risk	36
31.3 Short history of the current AI thematic basket	36
31.4 Economic-score specification	37
31.5 Coherence projection	37
31.6 Probability sample size	37
31.7 Conditional versus unconditional probability	37
31.8 Market-cap overlay stability	37
32 Reproducing the results	37
33 Interpretation and use	38
34 References	38

1 Model overview and key results

The model is designed around a simple risk-management question: *if an AI/TMT-led unwind occurs, how much loss can reach a position or client portfolio over a relevant risk horizon?* The question is conditional. It does not require the model to forecast the date of a market break before the model can be useful for margin, concentration, and counterparty-risk analysis.

The model is built around five main ideas.

1. **Market and thematic risk are separated.** Technology and AI securities have ordinary equity-market beta as well as incremental exposure to AI-specific repricing. The model represents these channels separately so the market move is not counted twice.
2. **Empirical exposure is the primary cross-sectional signal.** Statistical AI-excess intensity receives 60% weight. Economic relevance receives 40%, acting as a structural stabilizer rather than the dominant input.
3. **Exposure and severity are distinct.** I^{AI} answers *who is exposed*; S_X and κ answer *how severe the AI/TMT unwind is*. This distinction prevents a highly exposed company from mechanically changing the scenario itself.
4. **Speed of loss is historically anchored.** One-week through two-month stress magnitudes use actual fixed endpoints after the dot-com TMT peak, preserving the observed loss path instead of scaling a terminal drawdown by time.
5. **Probability is conditional on the adverse state.** The probability model allocates severity mass among the five cases once a qualifying burst has occurred. It intentionally does not infer an unconditional crash probability from the same small event sample.



Probability model conditions on burst severity; it does not forecast burst timing.

Figure 1: End-to-end model architecture. Exposure estimation, scenario severity, overlays, and portfolio application remain separately identifiable.

1.1 Key model parameters

Parameter	Model specification
Statistical / economic AI-intensity blend	60% statistical / 40% economic
Country / industry economic weighting	35% country / 65% industry
Country-industry tilt exponent	$\lambda = 0.50$
AI-intensity range	$0 \leq I^{AI} \leq 1.50$
High-AI coherence threshold	Economic industry score ≥ 0.50
Coherence margin	Country-index intensity $+0.05$
Base valuation severity	$\kappa_{Base} = 0.670739$
Scenario severity scalars	0.335, 0.671, 1.0, 1.5, 2.0
Risk horizons	5, 10, 21, 42 trading days
Probability conditioning threshold	Qualifying burst drawdown $D \geq 40\%$

The primary scalable output is a country-industry surface containing 27 country or regional buckets crossed with 71

client-facing industry labels, for 1,917 rows before scenario/horizon expansion. A separate approved single-stock layer handles concentrated names for which an industry average would be too coarse. Market capitalization is not used as a proxy for AI exposure; it enters only as a separate fragility overlay.

2 Model objective, scope, and conditioning

A useful stress test must state exactly what is conditioned on. Let $B = 1$ denote a qualifying bubble burst. The deterministic engine asks for the distribution of portfolio outcomes conditional on a specified severity path. The probability module asks how severity is distributed across the five representative cases conditional on $B = 1$.

The analysis separates three different questions:

$$\underbrace{\mathbb{P}(B = 1 \mid X_t)}_{\text{burst hazard / timing}} \quad , \quad \underbrace{\mathbb{P}(S = s \mid B = 1)}_{\text{conditional severity}} \quad , \quad \underbrace{Y_{s,h}(\text{portfolio})}_{\text{loss given severity}} .$$

The model estimates the second object and computes the third. It does not estimate the first. Keeping these questions separate matters. Crash severity can be measured from completed historical episodes, while the real-time probability and timing of a bubble break are much harder to identify and depend heavily on how a bubble is defined.

If an unconditional expected loss were ever required, it would need a separate hazard model:

$$\mathbb{E}[L_h] = \mathbb{P}(B = 1) \sum_s \mathbb{P}(S = s \mid B = 1) L_{s,h} + \mathbb{P}(B = 0) \mathbb{E}[L_h \mid B = 0] .$$

Nothing in the present conditional-severity calibration should be interpreted as supplying $\mathbb{P}(B = 1)$.

2.1 Assets and use cases

The model is designed for equity positions. It can be used to support client and portfolio diagnostics such as stressed P&L, adverse loss, stress loss relative to margin, and concentration review. Fixed-income and municipal exposures are not assigned an equity AI stress simply because they share an issuer name; they require separate risk models.

The model is transparent rather than optimized for a single forecasting statistic. Each final stress can be decomposed into a market contribution, an AI/TMT-excess contribution, and, where applicable, a size-fragility contribution. This decomposition makes the result easier to audit, explain, and challenge.

3 Data and calibration roles

The framework does not ask a single dataset to identify exposure, scenario size, structural relevance, liquidity fragility, and probability. Each data block has one statistical role. This separation helps avoid circular calibration: the same observations should not be used both to manufacture a high AI exposure and then to justify a larger AI shock.

Data block	Representative inputs	Statistical role
Current thematic returns	AI/semiconductor/software/cloud thematic baskets and approved stocks	Estimate raw AI co-movement and residual thematic diagnostics
Broad equity returns	World/broad equity proxy; S&P 500 for historical calibration	Estimate ordinary market beta and identify the broad crash leg
Country and industry proxies	Country ETFs, sector proxies, Fama–French industries	Estimate scalable country and industry exposure components
Economic relevance map	Independently specified country and client-industry scores	Structural prior for relationships too new or sparse to be measured cleanly in returns
Dot-com history	TMT industries, NASDAQ diagnostics, S&P 500	Historical analogue, terminal shock decomposition, and fixed-horizon loss path
Valuation observations	TMT and broad-market valuation multiples	Calibrate the Base fraction of the full historical AI/TMT-excess shock
Size portfolios	Fama–French market-equity portfolios and current breakpoints	Estimate generic small-cap fragility independently of AI exposure
Historical burst events	21 qualifying U.S. industry crash severities	Estimate $\mathbb{P}(S = s \mid B = 1)$ across the five severity cells
Position metadata	Country, industry, ticker, market cap, signed market value, optional margin	Select the appropriate stress row and convert return shock into P&L

The core empirical exposure layer is estimated at monthly frequency; the fixed-horizon crash path is measured at daily frequency. This is intentional. Monthly data stabilize cross-sectional beta estimation across heterogeneous securities, whereas daily data are necessary to preserve the speed and non-monotone shape of the historical post-peak loss path.

4 Core model

For country or region b , industry g , market-cap bucket m , severity case s , and horizon h , the full stress equation is

$$R_{b,g,m}^{s,h} = \beta_{b,g}^W S_W^{s,h} + I_{b,g}^{AI} S_X^{s,h} + A_m^{s,h}. \quad (1)$$

The variables have distinct economic meanings:

- $\beta_{b,g}^W$ is broad-market exposure;
- $S_W^{s,h}$ is the scenario/horizon broad-market log shock;
- $I_{b,g}^{AI}$ is non-negative incremental AI/TMT vulnerability after ordinary market beta is removed;
- $S_X^{s,h}$ is the incremental AI/TMT-excess log shock;
- $A_m^{s,h} \leq 0$ is a separate size-fragility log-return overlay;
- $R_{b,g,m}^{s,h}$ is the final stressed log return.

The use of log returns is structural rather than cosmetic. Independent multiplicative price effects become additive in log space, which allows market, thematic, and size channels to be summed once. The model converts to simple return only after the components have been combined:

$$q_{b,g,m}^{s,h} = \exp(R_{b,g,m}^{s,h}) - 1. \quad (2)$$

For a position i with signed market value MV_i , the signed stress P&L convention is

$$\Pi_i^{s,h} = -MV_i q_i^{s,h}. \quad (3)$$

For a long position, a negative stressed return creates a positive loss. For a short position, the same adverse equity move can create an economic gain. If an adverse-only risk metric is required, the model clips the signed P&L after calculating it:

$$L_{i,+}^{s,h} = \max(\Pi_i^{s,h}, 0). \quad (4)$$

This keeps the return model economically coherent while allowing conservative reporting conventions downstream.

Code implementation. Equation 1, the log-to-simple transformation, signed stress P&L, and the adverse-only reporting transform are implemented by the following standalone module.

```

1  """Canonical stress operator and return/P&L conversions."""
2  from __future__ import annotations
3  import numpy as np
4
5
6  def stress_log_return(beta_world, world_shock_log, ai_intensity,
7                      ai_excess_shock_log, size_overlay_log=0.0):
8      """R = beta*S_W + I_AI*S_X + A_size, all in log-return space."""
9      return (
10         np.asarray(beta_world, dtype=float) * float(world_shock_log)
11         + np.asarray(ai_intensity, dtype=float) * float(ai_excess_shock_log)
12         + np.asarray(size_overlay_log, dtype=float)
13     )
14
15
16  def simple_return(log_return):
17      """Exact conversion from log return to simple return."""
18      return np.exp(np.asarray(log_return, dtype=float)) - 1.0
19
20
21  def position_stress_pnl(signed_market_value_usd, stressed_simple_return):
22      """Positive number is loss; shorts may produce gains before adverse-only policy."""
23      return -np.asarray(signed_market_value_usd, dtype=float) * np.asarray(
24         stressed_simple_return, dtype=float
25     )
26
27
28  def adverse_only_loss(stress_pnl_usd):
29      """Reporting transform applied after signed economic P&L."""
30      return np.maximum(np.asarray(stress_pnl_usd, dtype=float), 0.0)

```

5 Return construction and statistical estimation

5.1 Monthly log-return frequency

The exposure layer uses monthly log returns. Daily data are useful for the short-horizon historical shock path, but monthly returns are preferable for cross-sectional factor estimation because they reduce microstructure noise and align securities with different trading calendars more robustly. For price $P_{j,t}$ at month end,

$$r_{j,t} = \log P_{j,t} - \log P_{j,t-1}. \quad (5)$$

5.2 Broad-market beta

For proxy j ,

$$r_{j,t} = \alpha_j + \beta_j^W r_{W,t} + u_{j,t}, \quad (6)$$

so

$$\hat{\beta}_j^W = \frac{\sum_t (r_{W,t} - \bar{r}_W)(r_{j,t} - \bar{r}_j)}{\sum_t (r_{W,t} - \bar{r}_W)^2}. \quad (7)$$

β^W represents the amount of broad-market repricing expected to transmit into the proxy before any additional AI-specific channel is introduced.

5.3 HAC inference and reliability

Monthly residuals can exhibit heteroskedasticity and serial dependence. The code therefore computes a Newey-West covariance estimate for diagnostics. The coefficient used by the stress model remains OLS; HAC affects inference and reliability assessment rather than changing the coefficient mechanically. With score vector $z_t = X_t u_t$ and Bartlett weights $w_\ell = 1 - \ell/(L+1)$, the long-run covariance estimator is

$$\hat{\Omega} = \sum_t z_t z_t' + \sum_{\ell=1}^L w_\ell \sum_t (z_t z_{t-\ell}' + z_{t-\ell} z_t'). \quad (8)$$

Then

$$\widehat{\text{Var}}(\hat{\beta}) = (X'X)^{-1} \hat{\Omega} (X'X)^{-1}. \quad (9)$$

When proxy histories are short or noisy, raw estimates can be reliability-weighted toward explicit priors. Importantly, raw coefficients, inference diagnostics, reliability weights, and final coefficients are stored separately. This avoids hiding estimation uncertainty inside a single opaque number.

Code implementation. The complete executable implementation of return construction, HAC OLS, AI orthogonalization, proxy exposure estimation, and optional reliability shrinkage follows.

```

1  """Return-estimation utilities for the AI-linked equity crash stress model.
2
3  This module implements the empirical layer used to estimate broad-market beta,
4  absolute co-movement with the raw AI basket, and the signed residual-AI loading
5  retained as a diagnostic. The stress engine consumes the resulting
6  coefficients; it does not re-fit regressions while stressing a portfolio.
7  """
8  from __future__ import annotations
9
10 from dataclasses import dataclass, asdict
11 import numpy as np
12 import pandas as pd
13
14 HAC_LAGS = 3
15 MIN_MONTHS = 24
16 MONTH_END = "ME"
17
18
19 @dataclass(frozen=True)
20 class HACRegressionResult:
21     alpha: float
22     beta: np.ndarray
23     se: np.ndarray
24     tstat: np.ndarray
25     residual: np.ndarray
26     r_squared: float
27     n_obs: int
28     nw_lags: int
29
30     def to_dict(self):
31         return asdict(self)
32
33
34 def month_end_log_returns(price: pd.Series) -> pd.Series:
35     """Convert a strictly positive daily price series to month-end log returns."""
36     p = pd.Series(price, copy=True).dropna().astype(float)
37     if not isinstance(p.index, pd.DatetimeIndex):
38         raise TypeError("price must have a DatetimeIndex")
39     if (p <= 0).any():
40         raise ValueError("prices must be strictly positive")
41     month_end = p.sort_index().resample(MONTH_END).last().dropna()
42     out = np.log(month_end).diff().dropna()
43     out.name = getattr(price, "name", None)
44     return out
45
46
47 def ols_hac(y, X, lags: int = HAC_LAGS) -> HACRegressionResult:
48     """OLS coefficients with Bartlett-kernel Newey-West covariance.
49
50     X must already include an intercept column. HAC changes inference, not the
51     OLS point estimate. This is deliberately explicit for auditability.
52     """
53     y = np.asarray(y, dtype=float).reshape(-1)
54     X = np.asarray(X, dtype=float)
55     mask = np.isfinite(y) & np.all(np.isfinite(X), axis=1)
56     y, X = y[mask], X[mask]
57     n, k = X.shape
58     if n <= k:
59         raise ValueError("insufficient observations")
60
61     beta = np.linalg.lstsq(X, y, rcond=None)[0]
62     residual = y - X @ beta
63     xtx_inv = np.linalg.pinv(X.T @ X)
64
65     # Newey-West 'meat'.
66     meat = np.zeros((k, k), dtype=float)
67     for t in range(n):
68         zt = X[t] * residual[t]
69         meat += np.outer(zt, zt)
70     L = min(int(lags), n - 1)
71     for ell in range(1, L + 1):
72         weight = 1.0 - ell / (L + 1.0)
73         for t in range(ell, n):
74             zt = X[t] * residual[t]
75             zlag = X[t - ell] * residual[t - ell]
76             meat += weight * (np.outer(zt, zlag) + np.outer(zlag, zt))
77
78     cov = xtx_inv @ meat @ xtx_inv
79     se = np.sqrt(np.maximum(np.diag(cov), 0.0))
80     tstat = np.divide(beta, se, out=np.full_like(beta, np.nan), where=se > 0)
81     centered = y - y.mean()
82     ss_tot = float(centered @ centered)
83     ss_res = float(residual @ residual)
84     r2 = np.nan if ss_tot <= 0 else 1.0 - ss_res / ss_tot
85     return HACRegressionResult(
86         alpha=float(beta[0]), beta=beta, se=se, tstat=tstat,
87         residual=residual, r_squared=float(r2), n_obs=n, nw_lags=L,
88     )
89
90
91 def orthogonal_ai_factor(ai_raw: pd.Series, world: pd.Series, lags: int = HAC_LAGS):
92     """Residualize the raw AI basket on the broad market with an intercept."""
93     d = pd.concat([ai_raw.rename("ai"), world.rename("world")], axis=1).dropna()
94     X = np.column_stack([np.ones(len(d)), d["world"].to_numpy(float)])
95     fit = ols_hac(d["ai"].to_numpy(float), X, lags=lags)
96     residual = pd.Series(fit.residual, index=d.index, name="ai_residual")
97     return fit, residual
98
99
100 def estimate_proxy_exposure(

```

```

101 proxy_return: pd.Series,
102 world_return: pd.Series,
103 ai_raw_return: pd.Series,
104 ai_residual_return: pd.Series,
105 lags: int = HAC_LAGS,
106 min_months: int = MIN_MONTHS,
107 ) -> dict:
108     """Estimate market beta, raw AI co-movement, and residual-AI diagnostic.
109
110     Three regressions are intentionally kept distinct:
111     proxy ~ 1 + world          -> beta_world
112     proxy ~ 1 + raw AI         -> theta_ai_abs
113     proxy ~ 1 + world + residual AI -> signed diagnostic gamma
114     """
115     d_ai = pd.concat([proxy_return.rename("y"), ai_raw_return.rename("ai")], axis=1).dropna()
116     d_w = pd.concat([proxy_return.rename("y"), world_return.rename("world")], axis=1).dropna()
117     d_r = pd.concat([
118         proxy_return.rename("y"), world_return.rename("world"),
119         ai_residual_return.rename("ai_resid")
120     ], axis=1).dropna()
121     if min(len(d_ai), len(d_w), len(d_r)) < min_months:
122         raise ValueError("insufficient common monthly observations")
123
124     fit_ai = ols_hac(d_ai["y"], np.column_stack([np.ones(len(d_ai)), d_ai["ai"]]), lags)
125     fit_w = ols_hac(d_w["y"], np.column_stack([np.ones(len(d_w)), d_w["world"]]), lags)
126     fit_r = ols_hac(
127         d_r["y"],
128         np.column_stack([np.ones(len(d_r)), d_r["world"], d_r["ai_resid"]]),
129         lags,
130     )
131     return {
132         "theta_ai_abs_raw": float(fit_ai.beta[1]),
133         "se_theta_ai_abs": float(fit_ai.se[1]),
134         "t_theta_ai_abs": float(fit_ai.tstat[1]),
135         "r2_ai_abs": fit_ai.r_squared,
136         "n_abs_months": fit_ai.n_obs,
137         "beta_world_raw": float(fit_w.beta[1]),
138         "se_beta_world": float(fit_w.se[1]),
139         "t_beta_world": float(fit_w.tstat[1]),
140         "r2_world": fit_w.r_squared,
141         "n_world_months": fit_w.n_obs,
142         "beta_world_in_residual_model": float(fit_r.beta[1]),
143         "gamma_ai_residual_raw": float(fit_r.beta[2]),
144         "se_gamma_ai_residual": float(fit_r.se[2]),
145         "t_gamma_ai_residual": float(fit_r.tstat[2]),
146         "r2_residual_model": fit_r.r_squared,
147         "n_residual_months": fit_r.n_obs,
148         "start": d_r.index.min(),
149         "end": d_r.index.max(),
150     }
151
152
153 def precision_shrink(raw_estimate, standard_error, prior_mean, prior_variance):
154     """Normal-normal empirical-Bayes precision shrinkage.
155
156     The reliability weight q approaches one for precise observations and zero
157     for noisy observations. Raw estimate, q, prior, and posterior are returned
158     separately so that shrinkage never becomes an invisible data transformation.
159     """
160     se2 = float(standard_error) ** 2
161     tau2 = max(float(prior_variance), 0.0)
162     if not np.isfinite(se2) or se2 < 0:
163         return {"estimate": float(prior_mean), "reliability_weight": 0.0}
164     if tau2 == 0:
165         return {"estimate": float(prior_mean), "reliability_weight": 0.0}
166     q = tau2 / (tau2 + se2)
167     est = q * float(raw_estimate) + (1.0 - q) * float(prior_mean)
168     return {"estimate": float(est), "reliability_weight": float(q)}

```

6 Orthogonalizing the AI thematic factor

A raw AI thematic basket is itself an equity portfolio and therefore contains broad-market exposure. If the raw AI basket were treated as an independent factor while a separate market factor were also applied, market risk could enter twice.

Let $r_{A,t}$ be the raw AI thematic-basket log return and $r_{W,t}$ the broad-market return. Project the AI basket on the market:

$$r_{A,t} = \alpha_A + \theta_W^{AI} r_{W,t} + x_t. \quad (10)$$

The residual

$$x_t = r_{A,t} - \hat{\alpha}_A - \hat{\theta}_W^{AI} r_{W,t} \quad (11)$$

is the direction of AI thematic movement that is not explained by contemporaneous broad-market movement. Under OLS with an intercept,

$$\sum_t (r_{W,t} - \bar{r}_W) x_t = 0, \quad (12)$$

up to numerical precision.

This orthogonalization serves two purposes. First, it provides a diagnostic residual factor that can be used to test whether a proxy moves with AI after ordinary equity beta is removed. Second, it motivates the beta-adjusted intensity

formula used in the model. The final stress model does not simply multiply a residual regression coefficient by a crash shock because a negative residual beta can mean relative outperformance, not an absolute gain, during a broad sell-off.

Interpretation and caveat. Negative residual AI beta should not be interpreted as a crash winner. A stock can outperform the AI basket while still fall materially through the market leg. The model therefore preserves the market term and constrains incremental AI vulnerability to be non-negative.

Code implementation. The exact projection operator and orthogonality diagnostic are reproduced below. The intercept is retained explicitly; this is important because sample orthogonality is with respect to centered market returns.

```

1  """Market/AI factor orthogonalization and projection diagnostics."""
2  from __future__ import annotations
3  import numpy as np
4
5
6  def ols_slope(y, x):
7      """OLS slope with an intercept: Cov(y,x)/Var(x)."""
8      y = np.asarray(y, dtype=float)
9      x = np.asarray(x, dtype=float)
10     mask = np.isfinite(y) & np.isfinite(x)
11     y, x = y[mask], x[mask]
12     if len(y) < 3:
13         return np.nan
14     xc, yc = x - x.mean(), y - y.mean()
15     den = float(xc @ xc)
16     return np.nan if den <= 0 else float((xc @ yc) / den)
17
18
19  def orthogonalize_ai(ai_returns, world_returns):
20      """Remove the fitted broad-market component from the raw AI basket.
21
22      r_A,t = alpha_A + theta_W^AI r_W,t + x_t.
23      With an intercept, x_t is orthogonal to centered r_W,t in sample.
24      """
25      ai = np.asarray(ai_returns, dtype=float)
26      world = np.asarray(world_returns, dtype=float)
27      mask = np.isfinite(ai) & np.isfinite(world)
28      ai, world = ai[mask], world[mask]
29      theta_world_ai = ols_slope(ai, world)
30      intercept = float(ai.mean()) - theta_world_ai * world.mean()
31      residual_ai = ai - intercept - theta_world_ai * world
32      return theta_world_ai, intercept, residual_ai
33
34
35  def orthogonality_diagnostic(world_returns, residual_ai):
36      """Return covariance, correlation and projection inner product."""
37      w = np.asarray(world_returns, dtype=float)
38      x = np.asarray(residual_ai, dtype=float)
39      mask = np.isfinite(w) & np.isfinite(x)
40      w, x = w[mask], x[mask]
41      wc = w - w.mean()
42      cov = float(np.mean(wc * (x - x.mean())))
43      corr = float(np.corrcoef(w, x)[0, 1]) if len(w) > 2 else np.nan
44      inner = float(wc @ x)
45      return {"covariance": cov, "correlation": corr, "centered_inner_product": inner}

```

7 Derivation of beta-adjusted statistical AI-excess intensity

7.1 Raw co-movement is not incremental exposure

Let θ_j^{abs} be the raw slope of proxy j on the raw AI basket. A large value can arise for two different reasons: the proxy has high ordinary market beta, or it has genuine incremental AI sensitivity. The statistical-intensity construction separates these channels.

Consider two benchmark assets. A pure broad-market exposure with market beta β_j^W should have AI-basket co-movement approximately $\beta_j^W \theta_W^{AI}$. A pure unit AI-epicenter exposure is normalized to have co-movement of one with the raw AI basket. An interpolation with incremental AI intensity I_j gives

$$\theta_j^{abs} \approx \beta_j^W \theta_W^{AI} + I_j (1 - \theta_W^{AI}). \quad (13)$$

Solving for I_j yields

$$I_j^{stat,raw} = \frac{\theta_j^{abs} - \beta_j^W \theta_W^{AI}}{1 - \theta_W^{AI}}. \quad (14)$$

The denominator is not arbitrary. It is the raw AI-basket co-movement available to the incremental thematic channel after accounting for the amount already embedded in the market direction. The normalization has useful limiting cases:

- if $\theta_j^{abs} = \beta_j^W \theta_W^{AI}$, then $I_j^{stat,raw} = 0$;
- if $\beta_j^W = 1$ and $\theta_j^{abs} = 1$, then $I_j^{stat,raw} = 1$;
- if raw AI co-movement is below the market-implied amount, $I_j^{stat,raw} < 0$, which is treated as relative defensiveness rather than a negative absolute crash exposure.

In the model,

$$I_j^{stat} = \min\{1.50, \max\{0, I_j^{stat,raw}\}\}. \quad (15)$$

The lower bound prevents the AI leg from mechanically producing gains during an AI crash; the upper bound limits unstable leverage from short or noisy return histories. Both the raw and clipped values are retained for review.

8 Economic relevance and the 60/40 blend

Historical return data cannot fully identify structural AI exposure. Some important relationships may be recent, may be obscured by conglomerate diversification, or may not have existed for a long enough sample. Conversely, a purely narrative score can be overly subjective. The model therefore uses the statistical score as the primary signal and economic relevance as a controlled stabilizer.

For a country-industry cell, let e_b denote country economic AI relevance and e_g industry economic relevance. The structural cell score is

$$I_{b,g}^{econ} = 0.35e_b + 0.65e_g. \quad (16)$$

Industry receives more weight because the direct business model generally determines thematic transmission more strongly than domicile, while country relevance still matters through supply-chain concentration, index composition, and macro/market structure.

Before the coherence adjustment, the AI intensity is

$$I_{b,g}^{pre} = 0.60I_{b,g}^{stat} + 0.40I_{b,g}^{econ}. \quad (17)$$

The 60/40 choice places greater weight on realized market evidence. Economic scoring cannot overwhelm a strong statistical signal, but it can prevent the model from treating a structurally central AI industry as unexposed merely because a proxy history is short or unusually noisy.

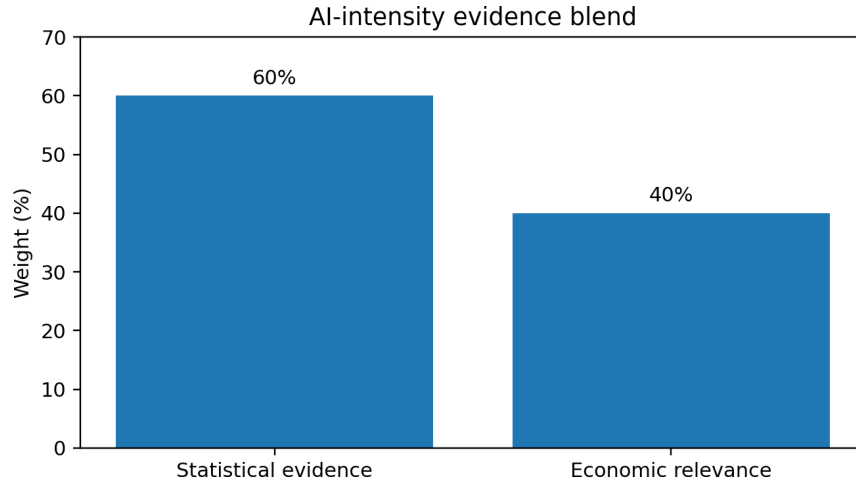


Figure 2: The final AI-vulnerability score places statistical evidence first and uses economic relevance as a secondary structural stabilizer.

8.1 Sensitivity of the blend

For fixed economic score I^{econ} , the marginal effect of the statistical estimate is

$$\frac{\partial I^{pre}}{\partial I^{stat}} = 0.60, \quad (18)$$

while

$$\frac{\partial I^{pre}}{\partial I^{econ}} = 0.40. \quad (19)$$

The blend is therefore transparent and monotone in both inputs. No nonlinear interaction is introduced at this stage. Nonlinearity is reserved for boundedness and the narrow coherence rule discussed below.

Code implementation. Equations 14, 15, 16, and 17 are implemented in one small module. The model constants in the source are 60% statistical, 40% economic, 35% country, 65% industry, and a 1.50 intensity cap.

```

1  """AI-exposure construction used by country, industry, and single-stock rows."""
2  from __future__ import annotations
3  import numpy as np
4
5  STATISTICAL_WEIGHT = 0.60
6  ECONOMIC_WEIGHT = 0.40
7  COUNTRY_ECONOMIC_WEIGHT = 0.35
8  INDUSTRY_ECONOMIC_WEIGHT = 0.65
9  INTENSITY_CAP = 1.50
10
11
12  def statistical_ai_excess_intensity(
13      theta_abs, beta_world, theta_world_ai,
14      lower: float = 0.0, upper: float = INTENSITY_CAP,
15  ):
16      """Beta-adjust raw AI co-movement into one-sided adverse AI intensity.
17
18      theta_abs -= beta_world*theta_world_ai
19                  + I_stat*(1-theta_world_ai)
20      hence
21      I_stat_raw = (theta_abs-beta_world*theta_world_ai)/(1-theta_world_ai).
22      """
23      den = 1.0 - float(theta_world_ai)
24      if abs(den) < 1e-12:
25          raise ValueError("theta_world_ai is too close to one for stable normalization")
26      raw = (float(theta_abs) - float(beta_world) * float(theta_world_ai)) / den
27      return float(raw), float(np.clip(raw, lower, upper))
28
29
30  def economic_ai_cell(country_score, industry_score):
31      """Structural score: 35% country, 65% industry."""
32      return float(
33          COUNTRY_ECONOMIC_WEIGHT * float(country_score)
34          + INDUSTRY_ECONOMIC_WEIGHT * float(industry_score)
35      )
36
37
38  def blend_ai_intensity(
39      statistical_intensity, economic_intensity,
40      statistical_weight: float = STATISTICAL_WEIGHT,
41      economic_weight: float = ECONOMIC_WEIGHT,
42      lower: float = 0.0, upper: float = INTENSITY_CAP,
43  ):
44      """Final empirical/structural blend: 60% statistical, 40% economic."""
45      if abs(statistical_weight + economic_weight - 1.0) > 1e-12:
46          raise ValueError("statistical and economic weights must sum to one")
47      pre = (
48          statistical_weight * float(statistical_intensity)
49          + economic_weight * float(economic_intensity)
50      )
51      return float(np.clip(pre, lower, upper))
52
53
54  def stock_ai_intensity(beta_world, theta_abs, theta_world_ai, economic_score):
55      """Convenience wrapper used by approved ticker overrides."""
56      raw, stat = statistical_ai_excess_intensity(theta_abs, beta_world, theta_world_ai)
57      final = blend_ai_intensity(stat, economic_score)
58      return {
59          "stat_ai_excess_intensity_raw": raw,
60          "stat_ai_excess_intensity": stat,
61          "economic_ai_intensity": float(economic_score),
62          "statistical_weight": STATISTICAL_WEIGHT,
63          "economic_weight": ECONOMIC_WEIGHT,
64          "final_ai_excess_intensity": final,
65      }

```

9 Identification as a change of factor basis

The preceding construction can be written more formally as a change of basis in return space. Ignoring intercepts for notational clarity, define the raw factor vector

$$f_t^{raw} = \begin{bmatrix} r_{W,t} \\ r_{A,t} \end{bmatrix}.$$

The orthogonalized factor vector is

$$f_t^{orth} = \begin{bmatrix} r_{W,t} \\ x_t \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\theta_W^{AI} & 1 \end{bmatrix} \begin{bmatrix} r_{W,t} \\ r_{A,t} \end{bmatrix} = T f_t^{raw}.$$

Since $\det(T) = 1$, the transformation is invertible:

$$f_t^{raw} = T^{-1} f_t^{orth} = \begin{bmatrix} 1 & 0 \\ \theta_W^{AI} & 1 \end{bmatrix} \begin{bmatrix} r_{W,t} \\ x_t \end{bmatrix}.$$

This matters because the model is not discarding information from the raw AI basket. It is re-expressing the same two-dimensional factor space in coordinates that distinguish ordinary market variation from incremental thematic variation.

Suppose a proxy admits the raw-factor representation

$$r_{j,t} = a_j + b_{j,W}^{raw} r_{W,t} + b_{j,A}^{raw} r_{A,t} + \varepsilon_{j,t}.$$

Substituting $r_A = \theta_W^{AI} r_W + x$ gives

$$r_{j,t} = a_j + (b_{j,W}^{raw} + \theta_W^{AI} b_{j,A}^{raw}) r_{W,t} + b_{j,A}^{raw} x_t + \varepsilon_{j,t}.$$

Thus the market coefficient in the orthogonal basis absorbs the market component embedded in the raw AI basket, while the residual-AI coefficient measures incremental co-movement. This is the same partial-regression logic formalized by the Frisch-Waugh-Lovell theorem.

The intensity in Equation 14 is not identical to the raw residual regression coefficient because the stress engine imposes a specific benchmark normalization. It asks: relative to a broad-market asset and a unit AI-epicenter asset, where does the observed raw co-movement place proxy j after controlling for its market beta? The result is an interpretable cross-sectional scale in which zero corresponds to market-implied AI co-movement and one corresponds to the normalized AI epicenter, before the 1.50 upper allowance for more concentrated names.

9.1 Why the model does not use a two-factor regression coefficient directly

A conventional two-factor regression could estimate

$$r_j = \alpha + \beta_W r_W + \gamma_X x + \varepsilon.$$

That is useful diagnostically, but using γ_X directly as a crash multiplier creates two practical issues. First, residual factors are estimated from normal and mixed market states, whereas the stress scenario is a one-sided adverse state. Second, a negative γ_X means the proxy tends to outperform the AI residual factor; it does not mean the proxy's absolute return should become positive when the market and AI sectors sell off together. The beta-adjusted intensity therefore preserves information about relative thematic exposure while enforcing a non-negative adverse thematic channel.

9.2 Identification assumptions

The construction relies on several explicit assumptions rather than hidden ones:

1. the broad-market proxy spans the main common equity direction relevant to the stress;
2. the AI thematic basket is sufficiently representative of current AI/TMT repricing to identify incremental co-movement;
3. monthly return relationships are informative about cross-sectional vulnerability even though scenario severity is calibrated from a historical crash analogue;
4. structural economic scores are specified independently and are not fitted to reproduce the same returns used to estimate I^{stat} ;
5. scenario shocks are exogenous conditional counterfactuals, so exposure estimation and scenario calibration can be performed separately.

These assumptions are reviewable. If a different market proxy, thematic basket, or structural map is adopted, the model can be rebuilt without changing the stress operator itself.

10 Country-industry lift and coherence projection

The scalable client vector requires a stress for each country-industry pair, but it is rarely possible to observe a liquid local ETF for every pair. The model therefore starts from an estimated country baseline and applies a tempered global industry tilt.

For country b and industry g ,

$$\beta_{b,g}^W = \beta_b^W (\max(\beta_g^W, \epsilon))^\lambda, \quad \lambda = 0.50, \quad (20)$$

and

$$\theta_{b,g}^{abs} = \theta_b^{abs} \left(\frac{\max(\theta_g^{abs}, \epsilon)}{\max(\theta_W^{AI}, \epsilon)} \right)^\lambda. \quad (21)$$

The square-root tilt prevents a global industry proxy from replacing the local country structure one-for-one. It changes the shape of the country baseline without claiming that the global industry ETF is a perfect local representation.

After computing $I_{b,g}^{stat}$ from Equation 14 and $I_{b,g}^{econ}$ from Equation 16, the model applies Equation 17. A narrow coherence projection is then applied only for explicitly AI-linked industries. If $e_g \geq 0.50$,

$$I_{b,g}^{AI} = \max(I_{b,g}^{pre}, I_{b,indx}^{AI} + 0.05), \quad (22)$$

subject to the overall 1.50 cap. Otherwise $I_{b,g}^{AI} = I_{b,g}^{pre}$.

The rationale is limited and specific: a directly AI-linked industry should not receive less incremental AI vulnerability than its own broad country index solely because the statistical industry tilt is noisy. This is not a broad manual override. The output stores whether the projection was applied and the applicable floor, allowing reviewers to isolate every affected row.

Code implementation. The complete country-industry construction code below implements the statistical intensity, 60/40 blend, 35/65 economic score, square-root country-industry lift, quality flags, and coherence projection.

```

1  """Country-industry latent exposure construction and coherence projection."""
2  from __future__ import annotations
3  import numpy as np
4  import pandas as pd
5
6  from ai_exposure import (
7      statistical_ai_excess_intensity, economic_ai_cell, blend_ai_intensity,
8      INTENSITY_CAP,
9  )
10
11  INDUSTRY_TILT = 0.50
12  AI_LINKED_ECON_THRESHOLD = 0.50
13  COHERENCE_MARGIN = 0.05
14
15
16  def latent_cell_exposure(
17      beta_country, beta_industry, theta_country, theta_industry,
18      theta_world_ai, industry_tilt: float = INDUSTRY_TILT,
19  ):
20      """Country baseline multiplied by a tempered global-industry tilt."""
21      beta_cell = float(beta_country) * max(float(beta_industry), 1e-12) ** industry_tilt
22      theta_ratio = max(float(theta_industry), 1e-12) / max(float(theta_world_ai), 1e-12)
23      theta_cell = float(theta_country) * theta_ratio ** industry_tilt
24      return float(beta_cell), float(theta_cell)
25
26
27  def coherence_project(
28      pre_intensity, country_index_intensity, industry_economic_score,
29      threshold: float = AI_LINKED_ECON_THRESHOLD,
30      margin: float = COHERENCE_MARGIN,
31      upper: float = INTENSITY_CAP,
32  ):
33      """AI-linked industries may not lie below broad-country intensity + margin."""
34      pre = np.asarray(pre_intensity, dtype=float)
35      idx = np.asarray(country_index_intensity, dtype=float)
36      econ = np.asarray(industry_economic_score, dtype=float)
37      floor = np.minimum(upper, idx + margin)
38      use_floor = econ >= threshold
39      projected = np.where(use_floor, np.maximum(pre, floor), pre)
40      applied = use_floor & (pre < floor)
41      return np.clip(projected, 0.0, upper), applied, floor
42
43
44  def build_country_industry_grid(country_estimates, industry_estimates, industry_mapping, theta_world_ai):
45      """Build all country x client-industry exposure rows.
46
47      Expected country columns: country_bucket, beta_world_final,
48      theta_ai_abs_final, country_economic_ai_intensity.
49      Expected industry columns: industry_bucket, beta_world_final,
50      theta_ai_abs_final.
51      Expected mapping columns: client_industry_label, industry_bucket,
52      client_industry_economic_ai_intensity.
53      """
54      countries = pd.DataFrame(country_estimates).copy()
55      industries = pd.DataFrame(industry_estimates).set_index("industry_bucket")
56      mapping = pd.DataFrame(industry_mapping).copy()
57      rows = []

```

```

58
59 for _, c in countries.iterrows():
60     beta_b = float(c["beta_world_final"])
61     theta_b = float(c["theta_ai_abs_final"])
62     econ_b = float(c["country_economic_ai_intensity"])
63     _, stat_b = statistical_ai_excess_intensity(theta_b, beta_b, theta_world_ai)
64     index_intensity = blend_ai_intensity(stat_b, econ_b)
65
66     for _, m in mapping.iterrows():
67         ib = m["industry_bucket"]
68         if ib not in industries.index:
69             continue
70         g = industries.loc[ib]
71         econ_g = float(m["client_industry_economic_ai_intensity"])
72
73         if str(ib).lower() in {"index or etf", "index & etf", "index etf"}:
74             beta_cell, theta_cell = beta_b, theta_b
75             _, stat_cell = statistical_ai_excess_intensity(theta_cell, beta_cell, theta_world_ai)
76             econ_cell = econ_b
77             pre = index_intensity
78             final = index_intensity
79             applied = False
80             floor = np.nan
81         else:
82             beta_cell, theta_cell = latent_cell_exposure(
83                 beta_b, float(g["beta_world_final"]),
84                 theta_b, float(g["theta_ai_abs_final"]), theta_world_ai,
85             )
86             raw_stat, stat_cell = statistical_ai_excess_intensity(
87                 theta_cell, beta_cell, theta_world_ai
88             )
89             econ_cell = economic_ai_cell(econ_b, econ_g)
90             pre = blend_ai_intensity(stat_cell, econ_cell)
91             final_a, applied_a, floor_a = coherence_project(
92                 np.array([pre]), np.array([index_intensity]), np.array([econ_g])
93             )
94             final, applied, floor = float(final_a[0]), bool(applied_a[0]), float(floor_a[0])
95
96         rows.append({
97             "country_bucket": c["country_bucket"],
98             "client_industry_label": m["client_industry_label"],
99             "industry_bucket": ib,
100             "beta_world_cell": beta_cell,
101             "theta_ai_abs_cell": theta_cell,
102             "theta_ai_abs_world_reference": float(theta_world_ai),
103             "cell_stat_ai_excess_intensity": stat_cell,
104             "country_economic_ai_intensity": econ_b,
105             "client_industry_economic_ai_intensity": econ_g,
106             "cell_economic_ai_intensity": econ_cell,
107             "country_index_final_ai_excess_intensity": index_intensity,
108             "final_ai_excess_intensity_pre_coherence": pre,
109             "monotonic_constraint_applied": applied,
110             "monotonic_floor_if_applicable": floor if econ_g >= AI_LINKED_ECON_THRESHOLD else np.nan,
111             "final_ai_excess_intensity": final,
112         })
113     return pd.DataFrame(rows)

```

11 Historical analogue selection and the role of dot-com

The historical technology unwind is used as a *shock calibration*, not as an assertion that the present market must follow the same calendar path. The analogue choice can nevertheless be tested rather than selected by eye. Let P_t^A denote the current AI thematic proxy and let $P_t^{D,\ell}$ denote a candidate historical dot-com segment ending ℓ trading days before the historical technology peak. Normalize both paths in cumulative log-price space,

$$x_t^A = \log(P_t^A / P_0^A), \quad x_t^{D,\ell} = \log(P_t^{D,\ell} / P_0^{D,\ell}).$$

For a comparison window of N observations, define

$$d(\ell) = \left[\frac{1}{N} \sum_{t=0}^{N-1} \left(x_t^A - x_t^{D,\ell} \right)^2 \right]^{1/2}. \quad (23)$$

The estimated alignment is

$$\hat{\ell} = \arg \min_{0 \leq \ell \leq 150} d(\ell).$$

Using a one-year trading window and a smoothed normalized path, the prior analogue-validation exercise located the minimum roughly 46 trading days before the historical dot-com peak. That result is descriptive evidence that the current AI path resembles a late-stage pre-peak technology run-up under this metric. It is *not* a forecast that the present market is 46 trading days from a peak.

The distinction matters. Path similarity supports the use of dot-com as a historical analogue for a conditional stress, but it is not a timing signal. The scenario does not use $\hat{\ell}$ as a clock, and the conditional probability model does not convert the alignment into $\Pr(B = 1)$.

Code implementation. The path-normalization, distance metric, and lead-search code are reproduced in full. The function returns the complete objective curve as well as the best-fit lead so that analogue choice can be challenged across the entire search range.

```

1  """Dot-com path-similarity diagnostic. This is evidence for analogue choice, not timing."""
2  from __future__ import annotations
3  import numpy as np
4  import pandas as pd
5
6
7  def normalize_log_path(price):
8      p = pd.Series(price, dtype=float).dropna()
9      if (p <= 0).any():
10         raise ValueError("price must be positive")
11     lp = np.log(p)
12     return lp - lp.iloc[0]
13
14
15  def rolling_smooth(values, window=20):
16      s = pd.Series(np.asarray(values, dtype=float))
17      return s.rolling(window, min_periods=1).mean().to_numpy()
18
19
20  def path_score(current_path, historical_path):
21      a = np.asarray(current_path, dtype=float)
22      b = np.asarray(historical_path, dtype=float)
23      n = min(len(a), len(b))
24      if n < 20:
25         raise ValueError("paths are too short")
26      a, b = a[-n:], b[-n:]
27      return {
28          "rmse": float(np.sqrt(np.mean((a - b) ** 2))),
29          "mae": float(np.mean(np.abs(a - b))),
30          "correlation": float(np.corrcoef(a, b)[0, 1]),
31      }
32
33
34  def fit_lead(current_price, reference_price, max_lead=150, n=252, smooth_window=20):
35      """Search historical segments ending 0..max_lead days before reference peak."""
36      current = pd.Series(current_price, dtype=float).dropna().iloc[-n:]
37      reference = pd.Series(reference_price, dtype=float).dropna()
38      x = rolling_smooth(normalize_log_path(current).to_numpy(), smooth_window)
39      peak = int(np.nanargmax(reference.to_numpy()))
40      rows = []
41      for lead in range(max_lead + 1):
42         end = peak - lead
43         start = end - n + 1
44         if start < 0:
45             continue
46         ref = reference.iloc[start:end + 1]
47         y = rolling_smooth(normalize_log_path(ref).to_numpy(), smooth_window)
48         score = path_score(x, y)
49         rows.append({"lead_trading_days": lead, **score, "start_index": start, "end_index": end})
50      if not rows:
51         raise ValueError("reference series is too short for requested search")
52      table = pd.DataFrame(rows).sort_values("lead_trading_days")
53      best = table.loc[table["rmse"].idxmin()].to_dict()
54      return best, table

```

12 Historical shock calibration: broad market versus AI/TMT excess

Exposure coefficients answer which assets are vulnerable. They do not determine how large the scenario should be. The shock side is calibrated separately.

Let S_E be the historical technology-epicenter log shock and S_W the contemporaneous broad-market log shock. The incremental TMT-excess shock is

$$S_X = S_E - S_W. \quad (24)$$

Thus

$$S_E = S_W + S_X. \quad (25)$$

This is the scenario-side analogue of factor orthogonalization: broad-market movement appears once, while only incremental technology repricing enters the thematic leg.

The technology epicenter is constructed from an equal-weight daily basket of Fama-French 49-industry Hardware, Software, Chips, and Telecommunications returns. The broad-market leg uses the S&P 500. This pairing provides a long historical technology-bubble analogue while keeping broad de-risking separately observable.

13 Valuation calibration and the Base severity scalar

A full dot-com TMT-excess replay is not used as the central Base case. Current valuation premium relative to the broad market is compared with the dot-com premium to determine what fraction of the historical excess repricing is applied.

For a TMT valuation multiple PE_{TMT} and broad-market multiple PE_W , define the log premium

$$P = \log \left(\frac{PE_{TMT}}{PE_W} \right). \quad (26)$$

Let P_{dotcom} be the reference dot-com log premium. The current blended premium is

$$P_{now} = 0.50P_{forward} + 0.25P_{spot} + 0.25P_{trailing}. \quad (27)$$

The raw severity ratio is

$$\kappa_{raw} = \frac{P_{now}}{P_{dotcom}}. \quad (28)$$

In the model, Base is constrained to the interval $[0.50, 1.00]$:

$$\kappa_{Base} = \min\{1, \max\{0.50, \kappa_{raw}\}\} = 0.670739. \quad (29)$$

This parameter scales only the AI/TMT-excess shock. It does not change I^{AI} . This distinction is central:

- I^{AI} is cross-sectional vulnerability estimated from current exposure evidence;
- κ is a scenario-severity parameter calibrated from valuation and historical analogues.

Code implementation. The valuation module implements the log-premium definition, 50/25/25 current-premium blend, constrained Base ratio, and an audit reproduction of the calibrated scalar. The historical spreadsheet ingestion can be refreshed independently; the equations used by the scenario engine remain fixed.

```

1  """Valuation-premium calibration of the Base AI/TMT-excess severity scalar."""
2  from __future__ import annotations
3  import math
4  import numpy as np
5  import pandas as pd
6
7  BASE_KAPPA_FLOOR = 0.50
8  BASE_KAPPA_CAP = 1.00
9
10
11 def weighted_geometric_mean(values, weights=None):
12     x = np.asarray(values, dtype=float)
13     if np.any(x <= 0):
14         raise ValueError("valuation multiples must be positive")
15     if weights is None:
16         weights = np.ones_like(x)
17     w = np.asarray(weights, dtype=float)
18     return float(np.exp(np.average(np.log(x), weights=w)))
19
20
21 def log_valuation_premium(ai_multiple, market_multiple):
22     """P = log(PE_AI / PE_market)."""
23     if ai_multiple <= 0 or market_multiple <= 0:
24         raise ValueError("valuation multiples must be positive")
25     return float(math.log(float(ai_multiple) / float(market_multiple)))
26
27
28 def blended_current_premium(forward_premium, current_premium, trailing_premium):
29     """50% forward, 25% current, 25% trailing log-premium blend."""
30     return float(
31         0.50 * float(forward_premium)
32         + 0.25 * float(current_premium)
33         + 0.25 * float(trailing_premium)
34     )
35
36
37 def base_severity_scalar(current_blended_premium, dotcom_reference_premium,
38                         lower=BASE_KAPPA_FLOOR, upper=BASE_KAPPA_CAP):
39     """Kappa_Base = current TMT premium / dot-com TMT premium, constrained to [0.5,1]."""
40     if dotcom_reference_premium <= 0:
41         raise ValueError("dot-com reference premium must be positive")
42     raw = float(current_blended_premium) / float(dotcom_reference_premium)
43     return {"kappa_raw": raw, "kappa_base": float(np.clip(raw, lower, upper))}
44
45
46 def reproduce_reference_calibration():
47     """Reference scalar values used by the model calibration."""
48     dotcom = 0.784787
49     current = 0.526387
50     out = base_severity_scalar(current, dotcom)
51     # Full-precision scalar is retained from the reference calibration table.
52     out["kappa_base_audited"] = 0.6707390702673786
53     out["dotcom_reference_premium_log"] = dotcom
54     out["current_blended_premium_log"] = current
55     return out
56
57
58 def premium_table_from_levels(df: pd.DataFrame):
59     """Utility for canonical tables containing ai_level and market_level columns."""
60     out = df.copy()
61     out["premium_log"] = np.log(out["ai_level"].astype(float) / out["market_level"].astype(float))
62     out["premium_ratio"] = np.exp(out["premium_log"])
63     return out

```

14 Five-case terminal scenario ladder

All five scenarios are points on the same structural model. They are not five unrelated narratives. The scenario specification is

Scenario	World scale	κ_s	Interpretation
Mild	0.50	0.335370	Half broad-market leg and half of Base AI/TMT-excess severity
Base	1.00	0.670739	Full market leg and valuation-calibrated AI/TMT-excess severity
Severe	1.00	1.000000	Full dot-com AI/TMT-excess analogue
Extreme	1.00	1.500000	Stress tail extension beyond the historical excess leg
Worst	1.00	2.000000	Deep tail extension of the AI/TMT-excess leg

Using the terminal market and historical excess references, the representative AI-epicenter drawdowns are approximately 31.9%, 53.7%, 63.9%, 75.3%, and 83.1%.

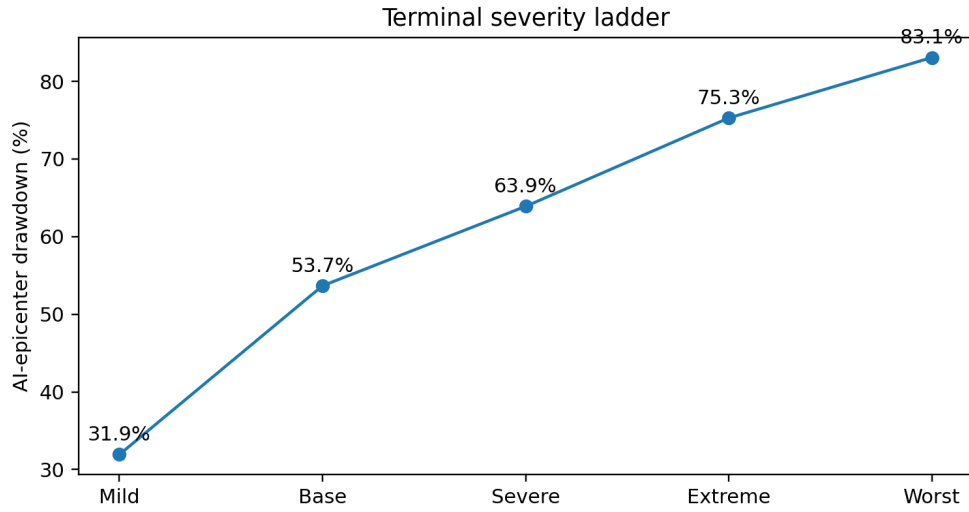


Figure 3: Terminal AI-epicenter severity ladder. Extreme and Worst extrapolate the excess channel beyond the full historical analogue.

Interpretation and caveat. Extreme and Worst are stress-design choices, not maximum-likelihood forecasts. Their purpose is to test nonlinear concentration and liquidity consequences under a materially more disorderly AI repricing than the historical dot-com excess leg.

15 Peak-anchored fixed-horizon mechanics

A terminal peak-to-trough drawdown answers a solvency-style question, but prime-brokerage and margin risk also depend on speed. The model therefore measures the historical loss after fixed numbers of trading days from the identified TMT peak.

Let t_0 denote the constructed TMT peak, 27 March 2000, and let t_h be the endpoint h trading observations later. Then

$$S_E^{raw,h} = \log \frac{P_E(t_h)}{P_E(t_0)}, \quad S_W^{raw,h} = \log \frac{P_W(t_h)}{P_W(t_0)}, \quad (30)$$

and

$$S_X^{raw,h} = S_E^{raw,h} - S_W^{raw,h}. \quad (31)$$

For scenario s ,

$$S_W^{s,h} = \omega_s S_W^{raw,h}, \quad S_X^{s,h} = \kappa_s S_X^{raw,h}, \quad (32)$$

with $\omega_{Mild} = 0.5$ and $\omega_s = 1$ otherwise.

15.1 Calibrated fixed-horizon surface

Horizon	Scenario	κ_s	World (%)	AI excess (%)	Epicenter (%)
1w	Mild	0.335	-0.59	-3.28	-3.84
1w	Base	0.671	-1.17	-6.44	-7.54
1w	Severe	1.000	-1.17	-9.45	-10.52
1w	Extreme	1.500	-1.17	-13.84	-14.85
1w	Worst	2.000	-1.17	-18.01	-18.97
2w	Mild	0.335	-0.64	-3.10	-3.72
2w	Base	0.671	-1.27	-6.11	-7.31
2w	Severe	1.000	-1.27	-8.97	-10.13
2w	Extreme	1.500	-1.27	-13.15	-14.26
2w	Worst	2.000	-1.27	-17.14	-18.19
1m	Mild	0.335	-2.08	-5.13	-7.11
1m	Base	0.671	-4.13	-10.00	-13.71
1m	Severe	1.000	-4.13	-14.53	-18.06
1m	Extreme	1.500	-4.13	-20.99	-24.25
1m	Worst	2.000	-4.13	-26.96	-29.97
2m	Mild	0.335	-4.78	-7.61	-12.03
2m	Base	0.671	-9.34	-14.64	-22.61
2m	Severe	1.000	-9.34	-21.02	-28.40
2m	Extreme	1.500	-9.34	-29.82	-36.37
2m	Worst	2.000	-9.34	-37.63	-43.45

The one-week endpoint is slightly more adverse than the two-week endpoint in the TMT excess component. This is not a model inconsistency. It reflects the actual path immediately after the peak: technology sold off sharply in the first week and partially retraced by the fixed ten-day endpoint. The one- and two-month endpoints subsequently become substantially more severe.

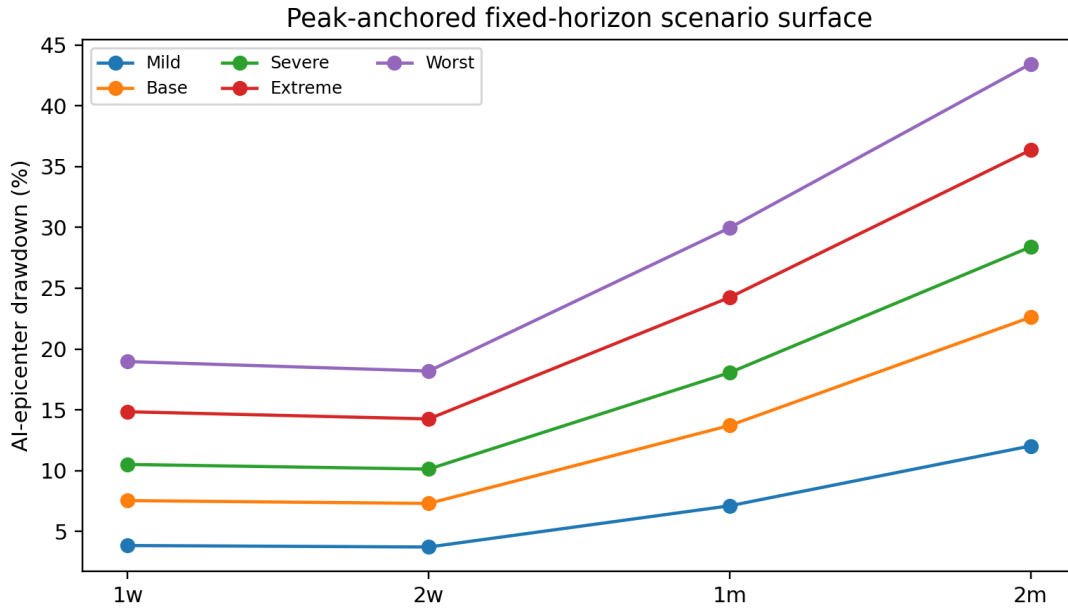


Figure 4: Fixed-horizon peak-anchored severity surface. The path is historical rather than mechanically time-scaled.

Code implementation. The complete scenario engine follows. It includes valuation calibration, terminal severity construction, fixed-horizon anchor derivation, scenario transformation, component attribution, and the deterministic stress operator.

```

1 """Historical shock decomposition, fixed-horizon anchors, and scenario surface."""
2 from __future__ import annotations
3 import numpy as np
4 import pandas as pd
5
6 KAPPA_BASE = 0.6707390702673786

```

```

7 SCENARIO_SPEC = {
8     "mild": {"world_scale": 0.50, "kappa": 0.5 * KAPPA_BASE},
9     "base": {"world_scale": 1.00, "kappa": KAPPA_BASE},
10    "severe": {"world_scale": 1.00, "kappa": 1.00},
11    "extreme": {"world_scale": 1.00, "kappa": 1.50},
12    "worst": {"world_scale": 1.00, "kappa": 2.00},
13 }
14 HORIZONS = {"1w": 5, "2w": 10, "1m": 21, "2m": 42}
15
16
17 def log_return_between(price: pd.Series, start_pos: int, end_pos: int):
18     p = pd.Series(price, dtype=float).dropna()
19     return float(np.log(p.iloc[end_pos] / p.iloc[start_pos]))
20
21
22 def decompose_historical_shock(epicenter_log, world_log):
23     """S_X = S_E - S_W so S_E = S_W + S_X exactly in log space."""
24     return {
25         "S_EPICENTER_log": float(epicenter_log),
26         "S_WORLD_log": float(world_log),
27         "S_AI_EXCESS_log": float(epicenter_log - world_log),
28     }
29
30
31 def peak_anchored_raw_horizons(epicenter_price, world_price, peak_date, horizons=HORIZONS):
32     """Measure actual post-peak endpoint shocks rather than linearly scaling terminal loss."""
33     e = pd.Series(epicenter_price, copy=True).dropna().sort_index()
34     w = pd.Series(world_price, copy=True).dropna().sort_index()
35     common = pd.concat([e.rename("epicenter"), w.rename("world")], axis=1).dropna()
36     t0 = common.index.get_indexer([pd.Timestamp(peak_date)], method="nearest")[0]
37     rows = []
38     for label, sessions in horizons.items():
39         end = t0 + sessions
40         if end >= len(common):
41             raise ValueError(f"insufficient history for {label}")
42         se = float(np.log(common["epicenter"].iloc[end] / common["epicenter"].iloc[t0]))
43         sw = float(np.log(common["world"].iloc[end] / common["world"].iloc[t0]))
44         rows.append({
45             "horizon": label, "sessions": sessions,
46             "peak_date": common.index[t0], "endpoint": common.index[end],
47             "S_EPICENTER_raw_log": se, "S_WORLD_raw_log": sw,
48             "S_AI_EXCESS_raw_log": se - sw,
49         })
50     return pd.DataFrame(rows)
51
52
53 def scenario_surface(raw_horizons: pd.DataFrame, scenario_spec=SCENARIO_SPEC):
54     """Apply scenario world-scale and kappa to every raw historical horizon."""
55     rows = []
56     for _, h in raw_horizons.iterrows():
57         for scenario, spec in scenario_spec.items():
58             sw = float(spec["world_scale"]) * float(h["S_WORLD_raw_log"])
59             sx = float(spec["kappa"]) * float(h["S_AI_EXCESS_raw_log"])
60             se = sw + sx
61             rows.append({
62                 "horizon": h["horizon"], "scenario": scenario,
63                 "world_scale": spec["world_scale"], "kappa": spec["kappa"],
64                 "S_WORLD_log": sw, "S_AI_EXCESS_log": sx,
65                 "S_AI_EPICENTER_log": se,
66                 "S_WORLD_simple": np.exp(sw) - 1.0,
67                 "S_AI_EXCESS_simple": np.exp(sx) - 1.0,
68                 "S_AI_EPICENTER_simple": np.exp(se) - 1.0,
69             })
70     return pd.DataFrame(rows)
71
72
73 def terminal_reference_scenarios():
74     """Exact terminal factor anchors inherited from the audited calibration."""
75     severe_world = -0.2612572967420284
76     severe_excess = -0.7576212009788291
77     rows = []
78     for scenario, spec in SCENARIO_SPEC.items():
79         sw = spec["world_scale"] * severe_world
80         sx = spec["kappa"] * severe_excess
81         se = sw + sx
82         rows.append({
83             "scenario": scenario, "world_scale": spec["world_scale"],
84             "valuation_severity_scalar": spec["kappa"],
85             "S_WORLD_log": sw, "S_AI_EXCESS_log": sx,
86             "S_AI_EPICENTER_log": se,
87             "S_WORLD_simple": np.exp(sw) - 1.0,
88             "S_AI_EXCESS_simple": np.exp(sx) - 1.0,
89             "S_AI_EPICENTER_simple": np.exp(se) - 1.0,
90         })
91     return pd.DataFrame(rows)
92
93
94 def stress_log_return(beta_world, ai_intensity, world_shock_log, ai_excess_shock_log):
95     return (
96         np.asarray(beta_world, dtype=float) * float(world_shock_log)
97         + np.asarray(ai_intensity, dtype=float) * float(ai_excess_shock_log)
98     )

```

16 Stress sensitivities, monotonicity, and nonlinear P&L

Equation 1 is linear in factor exposures and log shocks, which makes first-order sensitivity exact in log-return space. Before the size overlay,

$$R = \beta^W S_W + I^{AI} S_X.$$

Therefore

$$\frac{\partial R}{\partial \beta^W} = S_W, \quad \frac{\partial R}{\partial I^{AI}} = S_X.$$

In an adverse scenario $S_W < 0$ and $S_X < 0$, so increasing either market beta or AI vulnerability makes the log stress weakly more negative. This provides a direct monotonicity test for model outputs.

Because $S_X^{s,h} = \kappa_s S_X^{raw,h}$,

$$\frac{\partial R}{\partial \kappa_s} = I^{AI} S_X^{raw,h}.$$

The effect of moving from Base to Severe or from Severe to Extreme is therefore largest for rows with high I^{AI} , exactly as intended. A low-AI industry can still lose through $\beta^W S_W$, but changing AI severity should affect it much less than a semiconductor or AI-infrastructure row.

The conversion to simple return introduces the economically correct nonlinearity:

$$q = e^R - 1, \quad \frac{\partial q}{\partial R} = e^R = 1 + q > 0, \quad \frac{\partial^2 q}{\partial R^2} = e^R > 0.$$

For increasingly negative R , a fixed additional negative log shock produces a smaller absolute simple-return increment because simple return is bounded below by -100% . The model therefore never needs an arbitrary simple-return cap: the exponential mapping respects the natural price floor automatically.

For a long position with market value $MV > 0$,

$$L = -MV(e^R - 1) = MV(1 - e^R),$$

so

$$\frac{\partial L}{\partial R} = -MVe^R < 0.$$

The loss is increasing as R becomes more negative and is bounded above by the long position's market value. For a short position the signed P&L reverses mechanically. This is why signed economic P&L should be calculated before any adverse-only clipping convention.

16.1 Attribution under parameter changes

The linear log-space form also permits exact decomposition of model changes. If a row changes from (β_0, I_0) to (β_1, I_1) under the same scenario,

$$\Delta R = (\beta_1 - \beta_0)S_W + (I_1 - I_0)S_X.$$

If a scenario changes from $(S_{W,0}, S_{X,0})$ to $(S_{W,1}, S_{X,1})$ with fixed exposure,

$$\Delta R = \beta(S_{W,1} - S_{W,0}) + I(S_{X,1} - S_{X,0}).$$

These identities make it possible to tell whether a change in the final stress came from the exposure data or from the scenario calibration, without reverse-engineering the reported percentages.

17 Country-industry stress calculation and attribution

Once the exposure and shock layers have been built independently, the country-industry return before the size overlay is simply

$$R_{b,g}^{s,h} = \beta_{b,g}^W S_W^{s,h} + I_{b,g}^{AI} S_X^{s,h}. \quad (33)$$

The model stores the two components separately:

$$R_{b,g}^{W,s,h} = \beta_{b,g}^W S_W^{s,h}, \quad R_{b,g}^{X,s,h} = I_{b,g}^{AI} S_X^{s,h}. \quad (34)$$

Therefore

$$R_{b,g}^{s,h} = R_{b,g}^{W,s,h} + R_{b,g}^{X,s,h}. \quad (35)$$

This attribution is important for auditability. A reviewer can explain a final result by inspecting four quantities: the row's market beta, its final AI intensity, the selected market shock, and the selected AI-excess shock. A large loss must therefore be traceable to a large exposure coefficient, a large scenario shock, or both.

18 Single-stock override layer

The country-industry grid is the scalable default. For selected AI epicenter and supply-chain names, an industry average can dilute company-specific concentration. A stock override therefore applies the same model at ticker level:

$$R_q^{s,h} = \beta_q^W S_W^{s,h} + I_q^{AI} S_X^{s,h}. \quad (36)$$

The ticker's market beta, raw AI co-movement, statistical intensity, economic relevance, and final 60/40 AI intensity are estimated or retained from an approved coefficient table. The same scenario shock table is then applied. The override is a *replacement hierarchy*: if a valid ticker row exists, it supersedes the generic country-industry return. It is not added to the generic row and therefore does not double count AI exposure.

A representative terminal subset illustrates the logic:

Ticker	β^W	I^{stat}	I^{econ}	I^{AI}	Base	Worst
NVDA	1.455	1.500	1.00	1.300	-64.7%	-90.5%
AMD	1.981	1.460	0.90	1.236	-68.2%	-90.8%
AVGO	1.017	0.876	0.85	0.866	-50.6%	-79.3%
MSFT	0.851	0.489	0.75	0.593	-40.8%	-67.4%
TSM	1.142	0.667	1.00	0.800	-50.6%	-77.9%
2330.TW	1.058	0.492	1.00	0.695	-46.7%	-73.6%
000660.KS	1.224	1.202	1.00	1.121	-58.9%	-86.7%
005930.KS	0.950	0.472	0.90	0.643	-43.7%	-70.6%

These values are intended to show the mechanics: stock-level shocks can be materially more severe than country-industry averages because the ticker can have both a high market beta and a high incremental AI intensity.

Code implementation. The executable ticker-level estimation and scenario-materialization code follows.

```

1  """Ticker-level precision overrides using the same market + AI-excess model."""
2  from __future__ import annotations
3  from typing import Mapping
4  import numpy as np
5  import pandas as pd
6
7  from return_estimation import month_end_log_returns
8  from orthogonalization import ols_slope, orthogonalize_ai
9  from ai_exposure import stock_ai_intensity
10
11
12  def build_ai_factor(component_prices: Mapping[str, pd.Series], world_price: pd.Series):
13      """Aligned monthly world and equal-weight thematic basket returns."""
14      world = month_end_log_returns(world_price).rename("world")
15      components = [month_end_log_returns(p).rename(name) for name, p in component_prices.items()]
16      if not components:
17          raise ValueError("at least one AI thematic component is required")
18      comp = pd.concat(components, axis=1).dropna()
19      ai = comp.mean(axis=1).rename("ai")
20      factors = pd.concat([world, ai], axis=1).dropna()
21      theta_world_ai, intercept, residual = orthogonalize_ai(factors["ai"], factors["world"])
22      factors["ai_residual"] = residual
23      return factors, theta_world_ai
24
25
26  def estimate_stock_override(ticker, stock_price, factors, theta_world_ai, economic_ai_intensity):
27      stock = month_end_log_returns(stock_price).rename("stock")
28      d = pd.concat([stock, factors[["world", "ai", "ai_residual"]]], axis=1).dropna()
29      beta_world = ols_slope(d["stock"], d["world"])
30      theta_abs = ols_slope(d["stock"], d["ai"])
31      residual_beta = ols_slope(d["stock"], d["ai_residual"])
32      intensity = stock_ai_intensity(beta_world, theta_abs, theta_world_ai, economic_ai_intensity)
33      return {
34          "ticker": ticker,
35          "beta_world_stock": beta_world,
36          "theta_ai_abs_stock": theta_abs,
37          "theta_ai_abs_world_reference": theta_world_ai,
38          "residual_ai_beta_diagnostic": residual_beta,
39          "n_months": len(d),
40          **intensity,
41      }
42
43
44  def apply_scenarios(stock_table: pd.DataFrame, scenario_table: pd.DataFrame):
45      """Materialize scenario returns; ticker rows replace generic rows downstream."""
46      out = stock_table.copy()
47      for _, sc in scenario_table.iterrows():
48          s = str(sc["scenario"])
49          lr = (
50              out["beta_world_stock"].astype(float) * float(sc["S_WORLD_log"])
51              + out["final_ai_excess_intensity"].astype(float) * float(sc["S_AI_EXCESS_log"])
52          )
53          out[f"{s}_stress_log"] = lr
54          out[f"{s}_stress_simple"] = np.exp(lr) - 1.0
55          out[f"{s}_stress_pct"] = 100.0 * out[f"{s}_stress_simple"]
56      return out

```

19 Market-cap fragility as a separate risk channel

Market capitalization is not an AI-exposure variable. A small non-AI company is not automatically more AI-sensitive than a mega-cap semiconductor company. Size matters for a different reason: smaller firms can exhibit greater liquidity fragility, funding sensitivity, forced-selling pressure, and balance-sheet vulnerability during broad stress.

The model therefore adds size only after the market-plus-AI factor stress:

$$R_{b,g,m}^{s,h} = R_{b,g}^{s,h} + A_m^{s,h}. \quad (37)$$

The large/mega bucket is the reference and has $A_m^{s,h} = 0$ by construction.

19.1 Empirical calibration

For each fixed horizon, rolling log returns are constructed for Fama-French market-equity quintile portfolios and for the broad market. The calibration sample begins in 1986. The conditioning set consists of the bottom decile of broad-market rolling returns. Each smaller capitalization portfolio is measured relative to the top size quintile over the same stress windows.

Scenario overlays are selected from the conditional relative-return distribution:

Scenario	Conditional relative-return statistic
Mild	50th percentile
Base	25th percentile
Severe	10th percentile
Extreme	5th percentile
Worst	Historical minimum

Only adverse relative outcomes are retained:

$$A_m^{s,h} = \min\{Q_m^{s,h}, 0\}. \quad (38)$$

The market-cap breakpoints are refreshable empirical ranges rather than permanent economic constants. The current reference cutoffs are approximately \$1.15bn, \$3.37bn, \$7.56bn, and \$24.73bn between the five buckets.

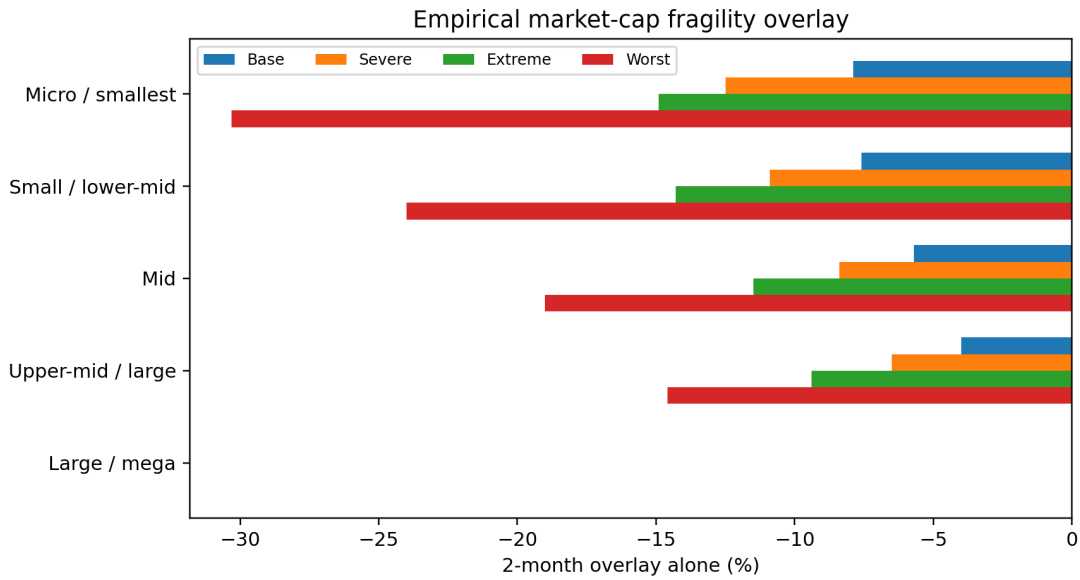


Figure 5: Reference two-month size overlays. The large/mega bucket is unchanged; smaller buckets receive additional loss calibrated independently of AI intensity.

Code implementation. The full market-cap fragility calibration code follows.

```
1 """Empirical market-cap fragility overlay calibrated from size portfolios."""
2 from __future__ import annotations
3 import numpy as np
4 import pandas as pd
5
6 SCENARIO_QUANTILES = {
7     "mild": 0.50,
8     "base": 0.25,
9     "severe": 0.10,
10    "extreme": 0.05,
11 }
12 HORIZONS = (5, 10, 21, 42)
13
14
15 def rolling_log_return(simple_return, window):
16     """Cumulative log return over a fixed window from simple daily returns."""
17     return np.log1p(pd.Series(simple_return, dtype=float)).rolling(window).sum()
18
19
20 def calibrate_size_fragility(
21     size_returns: pd.DataFrame,
22     broad_return: pd.Series,
23     horizons=HORIZONS,
24     stress_decile: float = 0.10,
25     reference_col: str = "Q5",
26 ):
27     """Estimate relative small-cap fragility in broad-market stress windows.
28
29     For each horizon, select bottom-decile broad-market rolling windows. For
30     every size bucket m, compute R_m - R_Q5 in those windows. Mild/Base/Severe/
31     Extreme use the 50/25/10/5 percentiles; Worst uses the historical minimum.
32     Positive relative outcomes are clipped to zero because the overlay is an
33     adverse fragility channel, not a source of stress relief.
34     """
35     size = pd.DataFrame(size_returns).copy()
36     broad = pd.Series(broad_return, copy=True).astype(float)
37     if reference_col not in size.columns:
38         raise ValueError(f"missing reference size portfolio {reference_col}")
39     rows = []
40     for h in horizons:
41         broad_roll = rolling_log_return(broad, h)
42         cutoff = float(broad_roll.quantile(stress_decile))
43         stressed = broad_roll <= cutoff
44         size_roll = pd.DataFrame({c: rolling_log_return(size[c], h) for c in size.columns})
45         rel = size_roll.sub(size_roll[reference_col], axis=0)
46         for bucket in size.columns:
47             sample = rel.loc[stressed, bucket].dropna().to_numpy(float)
48             if len(sample) == 0:
49                 raise ValueError(f"no stressed observations for horizon={h}, bucket={bucket}")
50             for scenario, q in SCENARIO_QUANTILES.items():
51                 overlay = min(float(np.quantile(sample, q)), 0.0)
52                 rows.append({"sessions": h, "cap_bucket": bucket, "scenario": scenario,
53                             "size_overlay_log": 0.0 if bucket == reference_col else overlay,
54                             "calibration_statistic": f"quantile_{q:.2f}"})
55             worst = min(float(np.min(sample)), 0.0)
56             rows.append({"sessions": h, "cap_bucket": bucket, "scenario": "worst",
57                         "size_overlay_log": 0.0 if bucket == reference_col else worst,
58                         "calibration_statistic": "historical_minimum"})
59     return pd.DataFrame(rows)
60
61
62 def add_size_overlay(base_stress_log, size_overlay_log):
63     return np.asarray(base_stress_log, dtype=float) + np.asarray(size_overlay_log, dtype=float)
64
65
66 def assign_market_cap_bucket(market_cap_usd, breakpoints: pd.DataFrame):
67     """Map current USD market capitalization to refreshable empirical ranges."""
68     value = float(market_cap_usd)
69     bp = breakpoints.sort_values("range_lower_usd").reset_index(drop=True)
70     for _, row in bp.iterrows():
71         lo = float(row["range_lower_usd"])
72         hi = row.get("range_upper_usd", np.nan)
73         if pd.isna(hi):
74             if value >= lo:
75                 return row["cap_bucket"]
76         elif lo <= value < float(hi):
77             return row["cap_bucket"]
78     raise ValueError("market cap outside supplied bucket ranges")
```

20 Portfolio application and loss aggregation

The final position-mapping hierarchy is deterministic:

1. normalize country and industry labels;
2. map the position to a country-industry exposure row;
3. select scenario s and horizon h ;
4. if an approved ticker override exists, replace the generic row;
5. if a market-cap bucket is supplied, add its independent fragility overlay;
6. convert the final log return to simple return once;
7. apply the simple return to signed market value;

8. aggregate signed and adverse-only losses as required.

For client margin M_c , a useful downstream metric is

$$\text{Stress loss to margin}_c = \frac{\sum_{i \in c} \max(\Pi_i, 0)}{M_c}. \quad (39)$$

This ratio is a portfolio output, not a calibration input. The model does not change asset stress coefficients to force a desired margin result.

Code implementation. The complete portfolio mapping and aggregation code follows.

```

1  """Position-level application hierarchy and stress P&L mechanics."""
2  from __future__ import annotations
3  import re
4  import numpy as np
5  import pandas as pd
6
7
8  def normalize_label(value):
9      return re.sub(r"[a-z0-9]+", " ", str(value).lower()).strip()
10
11
12  def normalize_ticker(value):
13      return re.sub(r"[A-Z0-9]+", "", str(value).upper())
14
15
16  def market_cap_bucket(market_cap_usd, breakpoints: pd.DataFrame):
17      value = float(market_cap_usd)
18      for _, row in breakpoints.sort_values("range_lower_usd").iterrows():
19          lo = float(row["range_lower_usd"])
20          hi = row.get("range_upper_usd", np.nan)
21          if (pd.isna(hi) and value >= lo) or (not pd.isna(hi) and lo <= value < float(hi)):
22              return row["cap_bucket"]
23      return breakpoints.iloc[-1]["cap_bucket"]
24
25
26  def apply_stress(
27      positions: pd.DataFrame,
28      country_industry: pd.DataFrame,
29      single_stock: pd.DataFrame,
30      country_map: pd.DataFrame,
31      size_breakpoints: pd.DataFrame | None = None,
32      size_overlays: pd.DataFrame | None = None,
33      scenario: str = "base",
34      horizon: str = "2m",
35  ):
36      """Map factors, replace with ticker overrides, add size, and compute P&L."""
37      pos = positions.copy()
38      required = ["country", "industry", "signed_market_value_usd"]
39      missing = [c for c in required if c not in pos.columns]
40      if missing:
41          raise ValueError(f"missing required position columns: {missing}")
42
43      pos["country_key"] = pos["country"].map(normalize_label)
44      pos["industry_key"] = pos["industry"].map(normalize_label)
45      pos["ticker_key"] = pos["ticker"].map(normalize_ticker) if "ticker" in pos else ""
46
47      cmap = country_map.copy()
48      cmap["country_key"] = cmap["client_country_label"].map(normalize_label)
49      pos = pos.merge(cmap[["country_key", "country_bucket"]], on="country_key", how="left")
50      pos["country_bucket"] = pos["country_bucket"].fillna("World fallback")
51
52      ci = country_industry.copy()
53      ci["industry_key"] = ci["client_industry_label"].map(normalize_label)
54      stress_col = f"{scenario}_stress_log"
55      cols = ["country_bucket", "industry_key", "beta_world_cell",
56             "final_ai_excess_intensity", stress_col]
57      pos = pos.merge(ci[cols], on=["country_bucket", "industry_key"], how="left")
58      pos["selected_stress_log"] = pos[stress_col]
59      pos["stress_source"] = "country_industry"
60
61      ss = single_stock.copy()
62      if len(ss):
63          ss["ticker_key"] = ss["ticker"].map(normalize_ticker)
64          sstress = f"{scenario}_stress_log"
65          lookup = ss.set_index("ticker_key")[ssstress].to_dict()
66          override = pos["ticker_key"].isin(lookup)
67          pos.loc[override, "selected_stress_log"] = pos.loc[override, "ticker_key"].map(lookup)
68          pos.loc[override, "stress_source"] = "single_stock_override"
69
70      pos["size_overlay_log"] = 0.0
71      if size_breakpoints is not None and size_overlays is not None and "market_cap_usd" in pos:
72          has_mc = pos["market_cap_usd"].notna()
73          pos.loc[has_mc, "cap_bucket"] = pos.loc[has_mc, "market_cap_usd"].map(
74              lambda x: market_cap_bucket(x, size_breakpoints)
75          )
76          ov = size_overlays.copy()
77          ov = ov[(ov["scenario"] == scenario) & (ov["horizon"] == horizon)]
78          ov_map = ov.set_index("cap_bucket")["size_overlay_log"].to_dict()
79          pos.loc[has_mc, "size_overlay_log"] = pos.loc[has_mc, "cap_bucket"].map(ov_map).fillna(0.0)
80
81      pos["final_stress_log"] = pos["selected_stress_log"] + pos["size_overlay_log"]
82      pos["stress_simple"] = np.exp(pos["final_stress_log"]) - 1.0
83      pos["stress_pct"] = 100.0 * pos["stress_simple"]
84      pos["stress_pnl_usd"] = -pos["signed_market_value_usd"] * pos["stress_simple"]
85      pos["adverse_stress_loss_usd"] = pos["stress_pnl_usd"].clip(lower=0.0)
86      if "client_level_margin_usd" in pos.columns:
87          pos["stress_loss_to_margin"] = (
88              pos["adverse_stress_loss_usd"] / pos["client_level_margin_usd"].replace(0.0, np.nan)
89          )
90      return pos

```

21 Probability model: objective and statistical object

The deterministic stress engine defines five severity cases but, by itself, does not say how likely each severity is conditional on a burst. The probability module adds this information without turning the stress framework into a market-timing model.

Let D be the maximum peak-to-trough drawdown after a qualifying bubble-like run-up. Define the burst event

$$B = 1 \iff D \geq 0.40. \quad (40)$$

The probability object is

$$\boxed{p_s = \mathbb{P}(S = s \mid B = 1)}, \quad (41)$$

where S is the representative scenario assigned to a realized continuous severity. The model does *not* estimate $\mathbb{P}(B = 1)$.

The five scenario anchors are approximately

$$(d_1, \dots, d_5) = (0.319, 0.537, 0.639, 0.753, 0.831). \quad (42)$$

A continuous drawdown has zero probability of exactly equaling any one of these points. The correct task is therefore to assign probability mass to the *region represented by each scenario*, rather than asking for the probability of an exact drawdown.

22 Probability geometry: positive log loss and Voronoi cells

Because the stress engine is additive in log-return space, the probability layer uses the same geometry. Define positive log loss

$$L = -\log(1 - D). \quad (43)$$

This transformation is monotone, maps $D \in [0, 1)$ to $L \in [0, \infty)$, and equals the magnitude of a negative log-return shock.

Let $L_s = -\log(1 - d_s)$ be the five scenario anchors in log-loss space. The boundary between adjacent representative points is the value b_s^L equidistant in squared log-loss distance:

$$(b - L_s)^2 = (b - L_{s+1})^2. \quad (44)$$

Expanding and cancelling b^2 gives

$$-2bL_s + L_s^2 = -2bL_{s+1} + L_{s+1}^2, \quad (45)$$

so

$$\boxed{b_s^L = \frac{L_s + L_{s+1}}{2}}. \quad (46)$$

Mapping back to drawdown space,

$$b_s^D = 1 - e^{-b_s^L}. \quad (47)$$

The resulting boundaries are approximately 43.8%, 59.1%, 70.1%, and 79.6%.

Conditional on $D \geq 40\%$, the cells are therefore

$$C_1 = [40.0\%, 43.8\%), \quad (48)$$

$$C_2 = [43.8\%, 59.1\%), \quad (49)$$

$$C_3 = [59.1\%, 70.1\%), \quad (50)$$

$$C_4 = [70.1\%, 79.6\%), \quad (51)$$

$$C_5 = [79.6\%, 100\%). \quad (52)$$

The Mild representative point itself is below 40%. This is not contradictory: conditional on a burst, Mild represents the least-severe burst region, not a claim that the realized drawdown must equal 31.9%.

23 Historical severity sample and empirical identification

The primary probability sample contains 21 exact U.S. industry crash episodes following qualifying run-ups. The observed drawdowns range from 44% to 75%, with mean approximately 60.3% and median 60.0%. The dot-com TMT subset is more severe on average, approximately 66%, but contains too few observations to support a stable standalone severity distribution. International evidence is used as an external reasonableness check rather than converted into pseudo-observations when exact episode-level drawdowns are not available.

Industry	Run-up first observed	Max. 24m drawdown
Automobiles and trucks	1928-03	53%
Chemicals	1929-01	67%
Electrical equipment	1929-01	55%
Utilities	1929-07	53%
Machinery	1936-05	55%
Steel works	1937-02	63%
Tobacco products	1961-11	44%
Real estate	1968-05	52%
Personal services	1968-05	69%
Entertainment	1972-05	60%
Restaurants and hotels	1972-06	55%
Precious metals	1979-12	48%
Petroleum and natural gas	1980-10	49%
Construction	1980-10	64%
Computer hardware	1999-03	68%
Computer software	1999-03	60%
Electronic equipment	1999-12	75%
Steel works	2000-08	66%
Measuring and control equipment	2000-02	61%
Steel works	2007-05	75%
Coal	2008-06	74%

The exact observations above are supplied in `data/historical_burst_severity_sample.csv`; the probability code reads this event-level sample directly.

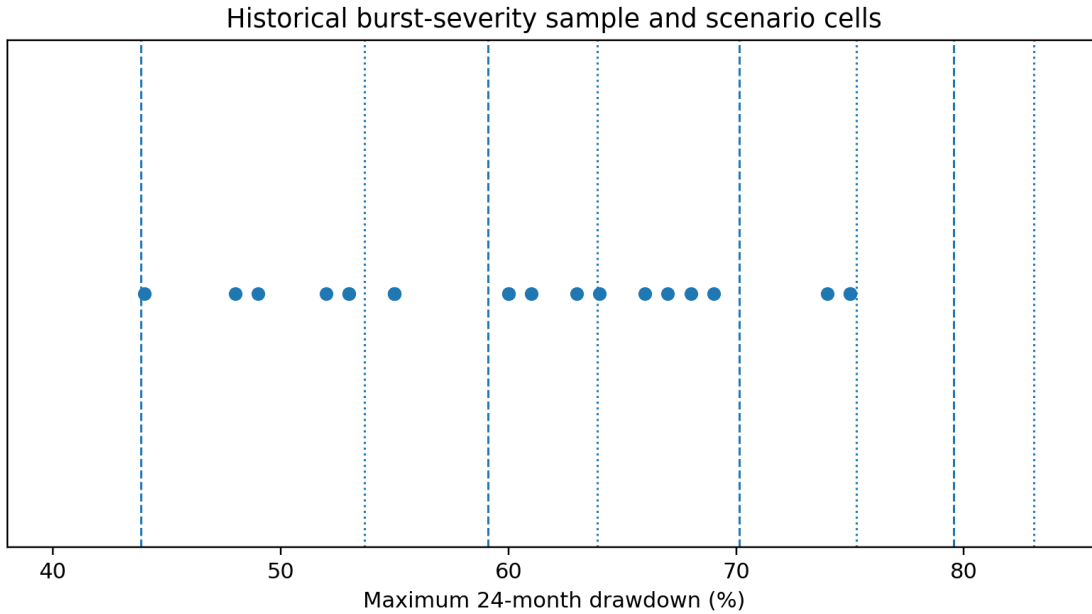


Figure 6: Historical burst severities relative to the five representative scenario anchors.

The small event sample is the central statistical constraint. It motivates three design decisions:

1. fit parsimonious two-parameter families rather than a high-dimensional distribution;

2. model-average plausible families instead of selecting one fragile winner;
3. quantify uncertainty by resampling whole historical episodes and refitting the entire ensemble.

24 Conditional severity distributions and likelihood construction

Four candidate families are used. They are specified on transformed variables that respect the burst threshold and the bounded nature of drawdowns.

24.1 Beta distribution on normalized drawdown

Define

$$Y = \frac{D - u}{1 - u}, \quad u = 0.40, \quad (53)$$

so $Y \in [0, 1)$. If $Y \sim \text{Beta}(a, b)$, the density of D includes the change-of-variable term

$$f_D(d) = \frac{1}{1 - u} f_{\text{Beta}}\left(\frac{d - u}{1 - u}; a, b\right). \quad (54)$$

24.2 Gamma, Weibull, and lognormal on excess log loss

Let

$$X = L - L_u = -\log(1 - D) + \log(1 - u), \quad (55)$$

where $L_u = -\log(1 - u)$. Then $X \geq 0$. Gamma and Weibull distributions naturally support positive X ; lognormal provides a positively skewed alternative. Because

$$\frac{dX}{dD} = \frac{1}{1 - D}, \quad (56)$$

the drawdown-space likelihood is

$$f_D(d) = f_X(x(d)) \frac{1}{1 - d}. \quad (57)$$

The Jacobian in Equation 57 is retained explicitly in the code. Omitting it would compare models on inconsistent measures.

Each family has two free parameters, which keeps the small-sample complexity penalty comparable.

25 AICc model averaging

For model m with maximized log likelihood ℓ_m , parameter count k_m , and sample size n , the small-sample Akaike criterion is

$$AICc_m = -2\ell_m + 2k_m + \frac{2k_m(k_m + 1)}{n - k_m - 1}. \quad (58)$$

The model weight is

$$w_m = \frac{\exp[-\frac{1}{2}(AICc_m - AICc_{\min})]}{\sum_j \exp[-\frac{1}{2}(AICc_j - AICc_{\min})]}. \quad (59)$$

The model-averaged burst-conditional CDF is

$$F^*(d \mid B = 1) = \sum_m w_m F_m(d \mid B = 1). \quad (60)$$

Scenario probability is the CDF mass within the corresponding Voronoi cell:

$$p_s = F^*(u_s) - F^*(\ell_s). \quad (61)$$

The fitted candidate models are close enough in AICc that hard selection would overstate confidence:

Model	AICc	Weight	$P(\text{Base})$	$P(\text{Severe})$	$P(\text{Extreme})$
Beta on normalized drawdown	-38.38	27.1%	47.6%	36.8%	12.3%
Gamma on excess log loss	-38.40	27.5%	47.5%	36.8%	12.3%
Weibull on excess log loss	-38.94	35.8%	43.4%	40.4%	13.2%
Lognormal on excess log loss	-36.32	9.7%	52.1%	31.6%	11.3%

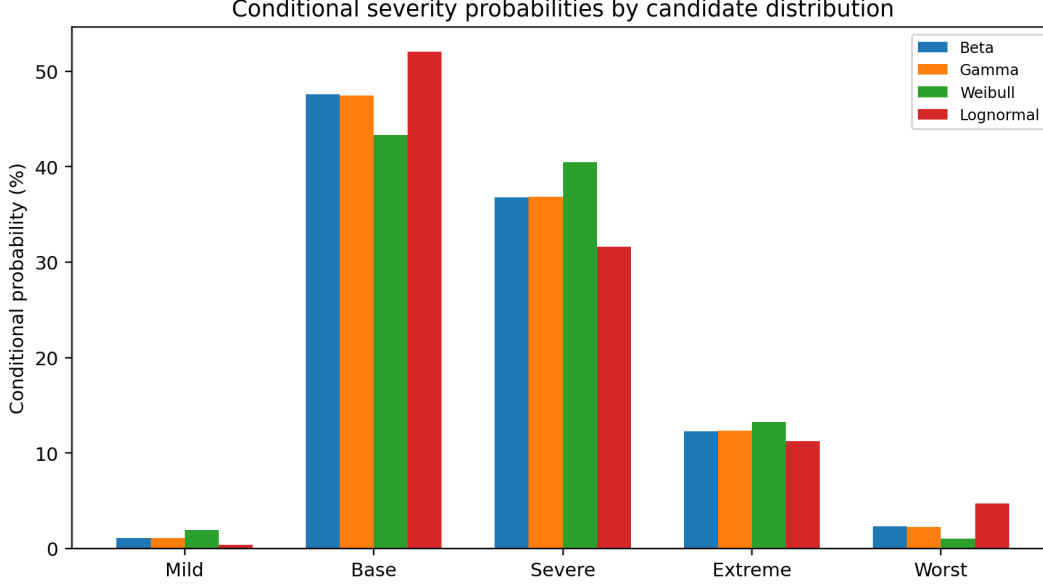


Figure 7: Candidate severity models produce similar central mass but differ enough to justify model averaging.

26 Bootstrap uncertainty, low-assumption cross-check, and tail diagnostic

26.1 Episode-level bootstrap

Parameter uncertainty and model-selection uncertainty are propagated by 5,000 nonparametric bootstrap replications. Each replication resamples the 21 historical episodes with replacement, refits all four candidate distributions, recomputes AICc weights, and recomputes the five cell probabilities. The reported interval is the 5th to 95th percentile of the resulting probability distribution for each case.

The procedure is episode-level. Individual return observations within a historical crash are not treated as independent severity events.

26.2 Jeffreys-prior Dirichlet cross-check

A low-assumption cross-check ignores the shape of the continuous distribution and uses only observed cell counts n_s . With a Jeffreys prior $\alpha_s = 1/2$, the posterior mean is

$$\tilde{p}_s = \frac{n_s + 1/2}{n + 5/2}. \quad (62)$$

This estimate is less smooth and cannot extrapolate the unobserved tail as effectively, but it is useful for showing that the central model result is not driven by a single parametric family.

26.3 Why generalized Pareto is diagnostic rather than baseline

Extreme Value Theory is a natural benchmark for threshold exceedances. However, with only 21 event-level severities, a free two-parameter generalized Pareto fit is highly unstable. In this sample its estimated finite upper endpoint is

approximately 75%, essentially the largest observed crash. Using that fit mechanically would assign no probability to a Worst region above roughly 80%, an implausibly strong inference from a very small sample. The GPD is therefore retained as a tail-model instability diagnostic, not as the main probability estimator.

27 Probability model details: estimators, quantization, and uncertainty

27.1 Maximum-likelihood parameterization

All four candidate distributions are estimated by maximum likelihood using support-respecting parameterizations. The Beta family is fitted on the normalized bounded variable $Y = (D - u)/(1 - u)$ with location fixed at zero and scale fixed at one. Gamma, Weibull, and lognormal families are fitted on positive excess log loss X with location fixed at zero. The implementation then evaluates every maximized likelihood back on the common drawdown measure D , including the change-of-variable Jacobian. This is the important comparability condition for AICc; the internal numerical parameterization of a distribution fit is otherwise immaterial to the model comparison.

For the normalized Beta model, if $y_i = (d_i - u)/(1 - u)$,

$$\ell_{Beta}(a, b) = \sum_i [(a - 1) \log y_i + (b - 1) \log(1 - y_i) - \log B(a, b) - \log(1 - u)].$$

For a transformed positive-loss family with density $g(x; \phi)$,

$$\ell(\phi) = \sum_i [\log g(x_i; \phi) - \log(1 - d_i)],$$

where the second term is the log Jacobian from Equation 57. Each fitted log likelihood is therefore comparable in the original drawdown measure before AICc is computed.

27.2 The five cases as a constrained scenario-reduction problem

If support points were free to move, one could choose five representative log losses to minimize a quantization objective such as

$$\mathcal{Q}(L_1, \dots, L_5) = \mathbb{E} \left[\min_s (L - L_s)^2 \mid B = 1 \right].$$

The model does not do this. The five scenario severities are fixed by the stress-design assumptions: Base is valuation-scaled, Severe is the full historical analogue, and Extreme/Worst are specified tail multiples. The probability task is therefore a *constrained scenario-reduction* problem: support points remain fixed and only the cell probabilities are estimated.

Given fixed ordered support, nearest-neighbor assignment under squared log-loss distance creates the midpoint boundaries in Equation 46. These cells are mutually exclusive and exhaustive, so

$$\sum_{s=1}^5 p_s = 1$$

by construction. This is preferable to overlapping arbitrary bands such as $\pm 10\%$ around each scenario, which would either double count probability mass or leave gaps.

27.3 Why AICc is used rather than ordinary AIC

With $n = 21$ and $k = 2$ per candidate family, the small-sample correction in Equation 58 is material relative to the differences among candidate models. Ordinary AIC is asymptotic; AICc adds a finite-sample penalty that vanishes as n becomes large. Because all four candidates have the same parameter count, the ranking remains close, but the reported criterion is appropriate to the actual sample size.

AICc weights are evidence weights within the candidate set, not posterior model probabilities under a fully specified Bayesian prior. Their role here is pragmatic: they prevent the final severity distribution from depending entirely on a single family when the sample does not strongly discriminate among plausible shapes.

27.4 Bootstrap uncertainty contains several sources at once

Each bootstrap replication repeats the full estimation workflow. It therefore incorporates variation from:

- which historical burst episodes appear in the resample;
- parameter estimation within each candidate family;
- changes in the relative AICc ranking of those families;
- changes in the model-averaged CDF mass crossing fixed scenario boundaries.

It does not incorporate structural uncertainty about the 40% burst definition, the choice of historical sample, or the five scenario anchors. Those are design sensitivities and should be tested separately.

27.5 Mean-preservation is a diagnostic, not an estimation target

The probability-weighted representative drawdown of approximately 60.6% is close to the observed 60.3% mean. The fitting procedure does not force this equality. It emerges from the fitted continuous severity distribution and fixed scenario cells. The closeness is therefore a useful ex-post diagnostic that the five-case representation preserves the central severity level of the historical sample.

A stronger validation program can compare additional moments and functionals, including median severity, tail probability above 70%, and expected positive log loss. With only 21 observations, however, increasingly detailed moment matching can create false precision. The current design prioritizes a small number of interpretable diagnostics.

27.6 Probability-weighted portfolio metrics are conditional summaries

Suppose $L_{s,h}$ is a client loss under deterministic case s . The quantity

$$\bar{L}_h^{burst} = \sum_s p_s L_{s,h}$$

is a conditional expected loss *given a burst*. It is useful for comparing portfolios with different cross-sectional exposures under the same adverse-state distribution. It is not a capital requirement by itself and should not replace explicit tail-case thresholds.

A particularly important nonlinear effect is that

$$\sum_s p_s [-MV(e^{R_s} - 1)] \neq -MV\left(e^{\sum_s p_s R_s} - 1\right)$$

in general. Probability weighting should therefore be applied to already-computed scenario P&Ls or simple-return outcomes, not by first averaging log shocks and then stressing the portfolio once.

28 Conditional scenario probabilities

The model-averaged probabilities are:

Scenario	Representative drawdown	Probability	5th pct.	95th pct.
Mild	31.9%	1.3%	0.1%	3.7%
Base	53.7%	46.5%	31.8%	59.9%
Severe	63.9%	37.6%	28.7%	48.5%
Extreme	75.3%	12.5%	5.5%	19.3%
Worst	83.1%	2.1%	0.4%	4.3%

Approximately 84% of conditional probability mass lies in Base plus Severe. The probability-weighted representative drawdown is about 60.6%, compared with a historical sample mean of about 60.3%. This is a useful severity-preservation check: the five-point discrete representation does not materially distort the mean of the underlying event sample.

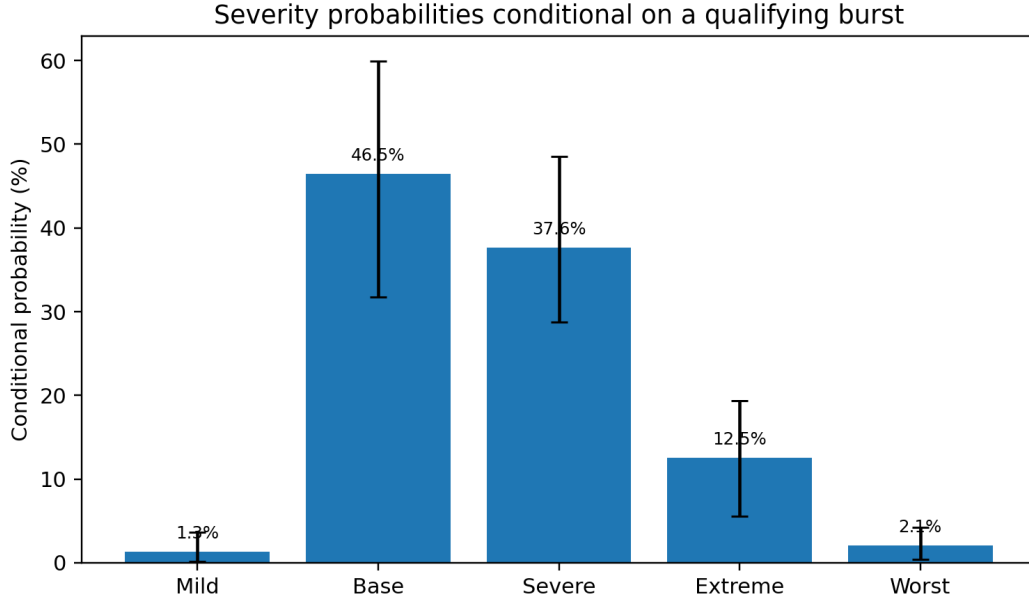


Figure 8: Final severity probabilities conditional on a qualifying burst, with 90% episode-level bootstrap intervals.

These probabilities should be used carefully. A 2.1% Worst weight does *not* mean there is a 2.1% chance of an 83% AI drawdown over the next month or year. It means that if a qualifying burst occurs, about 2.1% of the fitted conditional severity mass belongs to the region represented by the Worst case.

For any portfolio metric $Y_{s,h}$ already produced under each deterministic case,

$$\mathbb{E}[Y_h \mid B = 1] = \sum_{s=1}^5 p_s Y_{s,h}. \quad (63)$$

This conditional weighted metric is optional. Severe, Extreme, and Worst remain standalone stress tests and should not be weakened because their probability weights are smaller.

Code implementation. The complete burst-conditional severity implementation follows, including Voronoi cells, all four likelihoods with Jacobians, AICc weighting, full-ensemble bootstrap, Jeffreys-prior cross-check, GPD diagnostic, and conditional metric aggregation.

```

1  """Burst-conditional five-case severity probability model.
2
3  The object estimated here is P(S=s | B=1), where B=1 denotes a qualifying
4  bubble crash with maximum drawdown of at least 40%. This module DOES NOT
5  estimate P(B=1), the probability or timing of a burst.
6  """
7  from __future__ import annotations
8  from dataclasses import dataclass
9  import numpy as np
10 import pandas as pd
11 from scipy import stats
12
13 BURST_THRESHOLD = 0.40
14 BOOTSTRAP_REPLICATIONS = 5000
15 RANDOM_SEED = 20260805
16 SCENARIO_ORDER = ["mild", "base", "severe", "extreme", "worst"]
17 DEFAULT_ANCHORS = np.array([
18     0.31935317408616015,
19     0.5367198983734149,
20     0.6390004249225980,
21     0.7528325157387598,
22     0.8307705341123137,
23 ])
24
25
26 def positive_log_loss(drawdown):
27     d = np.asarray(drawdown, dtype=float)
28     if np.any((d < 0) | (d >= 1)):
29         raise ValueError("drawdown must lie in [0,1)")
30     return -np.log1p(-d)
31
32
33 def drawdown_from_log_loss(log_loss):
34     return 1.0 - np.exp(-np.asarray(log_loss, dtype=float))
35
36
37 def scenario_cells(anchors=DEFAULT_ANCHORS, burst_threshold=BURST_THRESHOLD):

```

```

38 """Nearest-representative Voronoi cells in positive log-loss space."""
39 d = np.asarray(anchors, dtype=float)
40 if len(d) != 5 or not np.all(np.diff(d) > 0):
41     raise ValueError("five strictly increasing scenario anchors are required")
42 L = positive_log_loss(d)
43 mid_L = 0.5 * (L[:-1] + L[1:])
44 mid_D = drawdown_from_log_loss(mid_L)
45 return pd.DataFrame({
46     "scenario": SCENARIO_ORDER,
47     "representative_drawdown": d,
48     "representative_log_loss": L,
49     "cell_lower_drawdown": np.array([burst_threshold, *mid_D]),
50     "cell_upper_drawdown": np.array([*mid_D, 1.0]),
51     "cell_lower_logloss": positive_log_loss(np.array([burst_threshold, *mid_D])),
52     "cell_upper_logloss": np.array([*mid_L, np.inf]),
53 })
54
55
56 def aicc(log_likelihood: float, parameter_count: int, sample_size: int) -> float:
57     """Small-sample corrected Akaike information criterion."""
58     k, n = float(parameter_count), float(sample_size)
59     if n <= k + 1:
60         return np.inf
61     return float(2*k - 2*log_likelihood + 2*k*(k+1)/(n-k-1))
62
63
64 def fit_models(drawdowns, burst_threshold=BURST_THRESHOLD):
65     """Fit Beta, Gamma, Weibull and Lognormal conditional severity families.
66
67     Beta is fitted to  $z=(D-u)/(1-u)$ . The other three are fitted to excess
68     positive log loss  $T=\log((1-u)/(1-D))$ . All log likelihoods are evaluated
69     on the common D scale, including the appropriate Jacobian, so AICc is
70     comparable across parameterizations.
71     """
72     D = np.asarray(drawdowns, dtype=float)
73     if np.any(D < burst_threshold) or np.any(D >= 1):
74         raise ValueError("conditional drawdowns must satisfy  $u \leq D < 1$ ")
75     n = len(D)
76     u = float(burst_threshold)
77     z = np.clip((D - u) / (1.0 - u), 1e-12, 1.0 - 1e-12)
78     T = np.maximum(np.log((1.0 - u) / (1.0 - D)), 1e-12)
79     fitted = []
80
81     beta_par = stats.beta.fit(z, floc=0, fscale=1)
82     beta_ll = float(np.sum(stats.beta.logpdf(z, *beta_par) - np.log(1.0 - u)))
83     def beta_cdf(d, par=beta_par):
84         d = np.asarray(d, dtype=float)
85         scaled = np.clip((d-u)/(1-u), 0.0, 1.0)
86         value = stats.beta.cdf(scaled, *par)
87         return np.where(d <= u, 0.0, np.where(d >= 1.0, 1.0, value))
88     fitted.append({"name": "Beta on normalized drawdown", "family": "Beta",
89                 "params": beta_par, "loglik": beta_ll, "k": 2,
90                 "aicc": aicc(beta_ll, 2, n), "cdf": beta_cdf})
91
92     for name, dist in [
93         ("Gamma on excess log-loss", stats.gamma),
94         ("Weibull on excess log-loss", stats.weibull_min),
95         ("Lognormal on excess log-loss", stats.lognorm),
96     ]:
97         par = dist.fit(T, floc=0)
98         #  $T(D)=\log((1-u)/(1-D))$ ;  $dT/dD = 1/(1-D)$ .
99         ll_D = float(np.sum(dist.logpdf(T, *par) - np.log(1.0 - D)))
100         def make_cdf(distribution, parameters):
101             def _cdf(d):
102                 d = np.asarray(d, dtype=float)
103                 clipped = np.minimum(d, 1.0 - 1e-12)
104                 t = np.log((1.0-u)/(1.0-clipped))
105                 value = distribution.cdf(np.maximum(t, 0.0), *parameters)
106                 return np.where(d <= u, 0.0, np.where(d >= 1.0, 1.0, value))
107             return _cdf
108         fitted.append({"name": name, "family": name.split()[0], "params": par,
109                     "loglik": ll_D, "k": 2, "aicc": aicc(ll_D, 2, n),
110                     "cdf": make_cdf(dist, par)})
111     return fitted
112
113
114 def model_average_probabilities(drawdowns, cells=None):
115     """Fit candidates, compute AICc weights, and integrate probability by cell."""
116     cells = scenario_cells() if cells is None else cells.copy()
117     fits = fit_models(drawdowns)
118     scores = np.array([f["aicc"] for f in fits], dtype=float)
119     raw = np.exp(-0.5 * (scores - scores.min()))
120     weights = raw / raw.sum()
121     lo = cells["cell_lower_drawdown"].to_numpy(float)
122     hi = cells["cell_upper_drawdown"].to_numpy(float)
123     per_model = []
124     for fit, weight in zip(fits, weights):
125         cdf = fit["cdf"]
126         p = np.array([float(cdf(hi[i]) - cdf(lo[i])) for i in range(5)])
127         p = np.maximum(p, 0.0)
128         p = p / p.sum()
129         fit["weight"] = float(weight)
130         fit["probabilities"] = p
131         per_model.append(p)
132     averaged = np.sum(np.asarray(per_model) * weights[:, None], axis=0)
133     averaged = averaged / averaged.sum()
134     return fits, averaged
135
136
137 def bootstrap_probabilities(
138     drawdowns, cells=None, replications=BOOTSTRAP_REPLICATIONS,
139     seed=RANDOM_SEED,
140 ):
141     """Episode-level nonparametric bootstrap with the full ensemble re-fitted."""
142     cells = scenario_cells() if cells is None else cells.copy()
143     observed = np.asarray(drawdowns, dtype=float)
144     rng = np.random.default_rng(seed)
145     accepted = []
146     attempts = 0
147     max_attempts = max(4*replications, replications+1000)
148     while len(accepted) < replications and attempts < max_attempts:

```

```

149     attempts += 1
150     sample = rng.choice(observed, size=len(observed), replace=True)
151     try:
152         _, p = model_average_probabilities(sample, cells)
153         if np.all(np.isfinite(p)) and abs(float(p.sum()) - 1.0) < 1e-8:
154             accepted.append(p)
155     except Exception:
156         continue
157     if len(accepted) < replications:
158         raise RuntimeError(f"only {len(accepted)} valid bootstrap replications")
159     return np.vstack(accepted)
160
161
162 def dirichlet_crosscheck(drawdowns, cells=None):
163     """Jeffreys-prior Dirichlet posterior mean for observed scenario-bin counts."""
164     cells = scenario_cells() if cells is None else cells.copy()
165     D = np.asarray(drawdowns, dtype=float)
166     lo = cells["cell_lower_drawdown"].to_numpy(float)
167     hi = cells["cell_upper_drawdown"].to_numpy(float)
168     counts = np.array([
169         np.sum((D >= lo[i]) & (D < hi[i])) if i < 4 else np.sum((D >= lo[i]) & (D <= hi[i]))
170         for i in range(5)
171     ], dtype=float)
172     posterior_alpha = counts + 0.5
173     return counts.astype(int), posterior_alpha / posterior_alpha.sum()
174
175
176 def gpd_diagnostic(drawdowns, burst_threshold=BURST_THRESHOLD):
177     """Unconstrained small-sample GPD diagnostic; not the main probability model."""
178     D = np.asarray(drawdowns, dtype=float)
179     uL = -np.log(1.0 - burst_threshold)
180     excess = -np.log(1.0 - D) - uL
181     shape, loc, scale = stats.genpareto.fit(excess, floc=0)
182     endpoint_log = np.inf if shape >= 0 else uL - scale / shape
183     endpoint_D = 1.0 if not np.isfinite(endpoint_log) else 1.0 - np.exp(-endpoint_log)
184     return {"shape": float(shape), "scale": float(scale), "finite_endpoint_drawdown": float(endpoint_D)}
185
186
187 def estimate(drawdowns, bootstrap=True):
188     """Return central estimates, model comparison, and optional bootstrap intervals."""
189     cells = scenario_cells()
190     fits, p = model_average_probabilities(drawdowns, cells)
191     result = cells.copy()
192     result["probability"] = p
193     counts, dmean = dirichlet_crosscheck(drawdowns, cells)
194     result["historical_count"] = counts
195     result["dirichlet_jeffreys_posterior_mean"] = dmean
196     if bootstrap:
197         boot = bootstrap_probabilities(drawdowns, cells)
198         q05, q50, q95 = np.quantile(boot, [0.05, 0.50, 0.95], axis=0)
199         result["bootstrap_p05"] = q05
200         result["bootstrap_median"] = q50
201         result["bootstrap_p95"] = q95
202     model_table = pd.DataFrame([
203         {"name": f["name"], "log_likelihood_D_space": f["loglik"],
204          "AICc": f["aicc"], "AICc_weight": f["weight"],
205          **{f"P_{SCENARIO_ORDER[i]}": f["probabilities"][i] for i in range(5)}}
206     ] for f in fits)
207     return result, model_table
208
209
210 def conditional_weighted_metric(values_by_scenario, probabilities):
211     """E[Y | B=1] = sum_s p_s Y_s; weight completed scenario outcomes, not log shocks."""
212     y = np.asarray(values_by_scenario, dtype=float)
213     p = np.asarray(probabilities, dtype=float)
214     if len(y) != 5 or len(p) != 5 or abs(p.sum()-1.0) > 1e-8:
215         raise ValueError("five outcomes and probabilities summing to one are required")
216     return float(p @ y)

```

29 Model validation

A stress framework should be challenged through explicit invariants rather than visual plausibility alone. The core checks are grouped into statistical, exposure, scenario, overlay, and portfolio identities.

29.1 Statistical invariants

- Orthogonalized AI residual has approximately zero sample covariance with centered broad-market returns.
- Raw coefficients, HAC diagnostics, reliability weights, and final coefficients remain separately stored.
- I^{stat} computed from Equation 14 is reproducible from stored θ^{abs} , β^W , and θ_W^{AI} .

29.2 Exposure invariants

- $0 \leq I^{AI} \leq 1.50$ for in-scope equity rows.
- Statistical and economic weights sum exactly to one.
- The economic country-industry score uses 35% country and 65% industry weights.

- Coherence can only increase an explicitly high-AI row to the stated country-index floor; it cannot alter a non-triggered row.

29.3 Scenario invariants

- $S_E = S_W + S_X$ in log space.
- For a fixed raw shock, Base through Worst use the same broad-market leg and increasing κ .
- Simple return equals $e^R - 1$ exactly up to floating-point precision.
- Country-industry total log stress equals market component plus AI component before size.

29.4 Overlay and mapping invariants

- Large/mega market-cap overlay equals zero.
- Smaller-cap overlays are non-positive after adverse clipping.
- A ticker override replaces the generic factor stress; it is not additive.
- Fixed-income/out-of-scope rows do not receive an equity AI stress by fallback.

29.5 Probability invariants

- Five probabilities are non-negative and sum to one conditional on $B = 1$.
- Voronoi cells are mutually exclusive and collectively exhaustive over [40%, 100%).
- Bootstrap refits resample events, not individual days within events.
- The probability layer does not alter deterministic scenario magnitudes.

Code implementation. The compact validation suite implementing the main numerical identities follows.

```

1  """Mechanical invariants for the AI-linked equity crash stress model."""
2  from __future__ import annotations
3  import numpy as np
4  import pandas as pd
5
6
7  def check_orthogonality(world, residual, tolerance=1e-10):
8      w = np.asarray(world, dtype=float); x = np.asarray(residual, dtype=float)
9      w = w - w.mean()
10     inner = float(w @ x)
11     return {"centered_inner_product": inner, "pass": abs(inner) <= tolerance}
12
13
14  def check_log_simple_identity(log_return, simple_return, tolerance=1e-12):
15     err = float(np.nanmax(np.abs(np.exp(np.asarray(log_return))-1.0-np.asarray(simple_return))))
16     return {"max_abs_error": err, "pass": err <= tolerance}
17
18
19  def check_component_identity(total_log, market_log, ai_log, size_log=0.0, tolerance=1e-12):
20     err = float(np.nanmax(np.abs(
21         np.asarray(total_log)-np.asarray(market_log)-np.asarray(ai_log)-np.asarray(size_log)
22     )))
23     return {"max_abs_error": err, "pass": err <= tolerance}
24
25
26  def check_ai_intensity_bounds(values, lower=0.0, upper=1.5, tolerance=1e-12):
27     x = np.asarray(values, dtype=float)
28     return {"minimum":float(np.nanmin(x)), "maximum":float(np.nanmax(x)),
29         "pass": bool(np.nanmin(x)>=lower-tolerance and np.nanmax(x)<=upper+tolerance)}
30
31
32  def check_probability_vector(probabilities, tolerance=1e-10):
33     p = np.asarray(probabilities, dtype=float)
34     ok = np.all(p >= -tolerance) and abs(float(p.sum())-1.0) <= tolerance
35     return {"sum":float(p.sum()), "minimum":float(p.min()), "pass":bool(ok)}
36
37
38  def check_large_cap_zero_overlay(size_overlay: pd.DataFrame, reference_bucket="Q5", tolerance=1e-12):
39     x = size_overlay.loc[size_overlay["cap_bucket"].eq(reference_bucket), "size_overlay_log"].to_numpy(float)
40     err = np.inf if len(x)==0 else float(np.max(np.abs(x)))
41     return {"max_abs_overlay":err, "pass":bool(err<=tolerance)}

```

30 Implementation and reproducibility

The body of this paper contains the executable implementation at the point where each theoretical mechanism is introduced. The following table provides a single cross-check that every core model block has an explicit source module.

Model mechanism	Executable source	Principal output object
Master log stress / simple return / P&L	<code>code/core_operator.py</code>	stressed log return, simple return, signed/adverse loss
Monthly returns, OLS/HAC, proxy coefficients	<code>code/return_estimation.py</code>	β^W , θ^{abs} , HAC diagnostics, residual γ
AI-on-market projection	<code>code/orthogonalization.py</code>	θ_W^{AI} , intercept, residual x_t
Statistical intensity and 60/40 blend	<code>code/ai_exposure.py</code>	$I^{stat,raw}$, I^{stat} , I^{econ} , I^{AI}
Country-industry latent lift / coherence	<code>code/country_industry.py</code>	scalable country-industry exposure grid
Dot-com analogue path diagnostic	<code>code/analogue_validation.py</code>	lead-distance objective and best-fit alignment
Valuation-to-severity calibration	<code>code/valuation_calibration.py</code>	κ_{Base}
Terminal and fixed-horizon scenario surface	<code>code/horizon_scenarios.py</code>	$S_W^{s,h}$, $S_X^{s,h}$, epicenter shock
Approved ticker replacement rows	<code>code/single_stock.py</code>	stock-specific β^W , I^{AI} , stress return
Market-cap fragility	<code>code/market_cap.py</code>	$A_m^{s,h}$ and cap-bucket mapping
Position selection / aggregation	<code>code/portfolio_application.py</code>	row selection, final return, stress P&L, loss-to-margin
Conditional severity probability	<code>code/probability_model.py</code>	p_s , AICc model table, bootstrap intervals, cross-checks
Mechanical audit	<code>code/validation.py</code>	invariant pass/fail diagnostics

The source modules are intentionally small. They encode the mathematical operators without report-generation, plotting, or file-packaging glue. This keeps the implementation that changes model values separate from code that merely formats an output. The bundle also contains the exact 21-event severity sample, terminal scenario table, short-horizon reference table, and the 5,000-replication probability bootstrap generated from the included estimator.

31 Model limitations and controls

31.1 Historical analogue risk

Dot-com is the clearest modern technology-led equity-bubble unwind, but the current AI ecosystem differs in profitability, index concentration, capital intensity, financing structure, and global supply chains. The analogue is therefore decomposed and scaled rather than replayed mechanically. Severe is the full historical excess analogue; Base is valuation-scaled; tail cases test extrapolations.

31.2 Proxy and basis risk

Country ETFs and global industry proxies are imperfect representations of local client positions. The country-industry tilt is intentionally tempered, and reliability metadata should be used to identify rows where a direct local proxy would materially improve estimation. Important single stocks are handled through explicit overrides rather than by globally increasing industry stress.

31.3 Short history of the current AI thematic basket

The rapid evolution of AI-specific securities means a clean thematic basket may have a shorter history than the broad market. This is one reason statistical evidence is combined with an economic score rather than treated as infallible. The 60/40 blend still keeps empirical evidence dominant.

31.4 Economic-score specification

Economic relevance is model input, not empirical fact. Scores should be documented, reviewed on a controlled schedule, and changed only when the underlying transmission mechanism changes. They should not be adjusted to produce a desired portfolio loss. Because the statistical weight is 60%, moderate score changes have bounded direct impact on I^{AI} .

31.5 Coherence projection

The monotonicity floor is a model constraint, not an estimated parameter. It should remain narrow, auditable, and limited to explicitly AI-linked industries. A large share of rows being affected by the floor would indicate that the underlying statistical or proxy layer needs review rather than that the floor should be expanded.

31.6 Probability sample size

Twenty-one event-level U.S. crash observations are sufficient for an informative conditional severity distribution but not for precise tail inference. Wide bootstrap intervals are therefore a feature, not a defect. The model ensemble reduces single-family dependence, but it cannot create information absent from the historical sample.

31.7 Conditional versus unconditional probability

The probability weights are not calendar-horizon crash frequencies. Any future attempt to estimate a burst hazard should remain a separate module and could consider run-up magnitude, volatility, turnover, issuance, valuation, market breadth, option-implied measures, or price-path acceleration. Combining a hazard model with the present conditional severity distribution should happen only after independent validation of the hazard model.

31.8 Market-cap overlay stability

Size breakpoints and relative stress distributions can change as the listed-company universe evolves. The calibration should therefore be refreshed periodically. The large/mega zero-reference convention should remain fixed so changes in the overlay remain interpretable as relative size fragility rather than changes in the base factor stress.

32 Reproducing the results

The model can be reproduced in a controlled sequence with no circular dependence:

1. Construct monthly broad-market, AI thematic, country, industry, and approved stock returns.
2. Orthogonalize the raw AI thematic return against the broad market and store θ_W^{AI} .
3. Estimate β^W and raw θ^{abs} for country, industry, and stock proxies; retain HAC diagnostics.
4. Apply any reliability shrinkage to create final country/industry coefficients while retaining raw values.
5. Compute beta-adjusted I^{stat} from Equation 14.
6. Build country-industry economic scores and apply the 60/40 blend.
7. Apply the narrow coherence projection and store its audit flag.
8. Reconstruct the dot-com market/TMT-excess shock decomposition.
9. Calibrate κ_{Base} from valuation premiums and build the five terminal cases.
10. Build 1w/2w/1m/2m fixed-endpoint historical shock anchors and apply the five scenario scalars.
11. Materialize country-industry and approved stock stress surfaces with component attribution.
12. Calibrate and apply the independent market-cap fragility surface.

13. Map positions using the country-industry default and ticker replacement hierarchy.
14. Convert final log return to simple return once and compute signed/adverse P&L.
15. Fit the conditional severity ensemble, compute AICc weights, bootstrap probability uncertainty, and store the five p_s .
16. Run the validation invariants before publishing outputs.

The bundle is organized so that the model blocks above correspond directly to executable Python modules. The reference input and output CSVs provide numerical anchors for independent implementation checks.

33 Interpretation and use

The framework should be read as a structured set of conditional questions rather than a single forecast.

Base is the central valuation-scaled unwind: full broad-market de-risking plus approximately 67% of the historical dot-com TMT-excess channel. **Severe** is the full historical excess analogue. **Extreme** and **Worst** are deliberate tail extensions that test concentration, liquidity, and margin resilience if the AI-specific repricing is materially larger than dot-com.

Cross-sectional differentiation comes from two evidence layers. The 60% statistical component uses observed return co-movement after removing ordinary market beta. The 40% economic component captures structural exposure that may not yet be fully represented in return history. The result is a model that is empirically anchored without treating a short, rapidly changing AI history as complete.

Probability weights provide a disciplined way to summarize conditional severity, but they should not replace standalone tail stress. In particular, the Base-versus-Severe split carries meaningful statistical uncertainty. Decisions sensitive to client solvency, margin adequacy, or concentrated single-name exposures should therefore be reviewed under Severe and Extreme directly even when a probability-weighted conditional metric is also shown.

The most important interpretive principle is separation of mechanisms. Broad-market beta explains ordinary equity de-risking. AI intensity explains incremental thematic repricing. Market cap explains generic size fragility. Scenario severity controls the strength of the adverse state. Probability describes the distribution of severity after the adverse state occurs. Keeping these functions separate is what makes the framework auditable and extensible.

34 References

1. Fama, E. F. and French, K. R. (1993), “Common Risk Factors in the Returns on Stocks and Bonds,” *Journal of Financial Economics*.
2. Fama, E. F. and French, K. R. (1997), “Industry Costs of Equity,” *Journal of Financial Economics*.
3. Frisch, R. and Waugh, F. V. (1933), “Partial Time Regressions as Compared with Individual Trends,” *Econometrica*.
4. Lovell, M. C. (1963), “Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis,” *Journal of the American Statistical Association*. The Frisch-Waugh-Lovell result motivates the residualization interpretation used in factor orthogonalization.
5. Newey, W. K. and West, K. D. (1987), “A Simple, Positive Semi-definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix,” *Econometrica*.
6. Greenwood, R., Shleifer, A., and You, Y. (2019), “Bubbles for Fama,” *Journal of Financial Economics*, 131, 20–43.
7. McNeil, A. J. and Frey, R. (2000), “Estimation of Tail-Related Risk Measures for Heteroscedastic Financial Time Series: An Extreme Value Approach,” *Journal of Empirical Finance*.
8. Burnham, K. P. and Anderson, D. R. (2002), *Model Selection and Multimodel Inference*, Springer. The probability layer uses the small-sample AICc and model-weighting logic described in this literature.
9. Efron, B. and Tibshirani, R. J. (1993), *An Introduction to the Bootstrap*, Chapman & Hall.
10. Pickands, J. (1975), “Statistical Inference Using Extreme Order Statistics,” *Annals of Statistics*, 3, 119–131.

11. Balkema, A. A. and de Haan, L. (1974), “Residual Life Time at Great Age,” *Annals of Probability*, 2, 792–804.
12. Dupačová, J., Gröwe-Kuska, N., and Römisch, W. (2003), “Scenario Reduction in Stochastic Programming: An Approach Using Probability Metrics,” *Mathematical Programming*, 95, 493–511.
13. Heston, S. L. and Rouwenhorst, K. G. (1994), “Does Industrial Structure Explain the Benefits of International Diversification?” *Journal of Financial Economics*, 36, 3–27.
14. Griffin, J. M. and Karolyi, G. A. (1998), “Another Look at the Role of Industrial Structure of Markets for International Diversification Strategies,” *Journal of Financial Economics*, 50, 351–373.
15. Phillips, P. C. B., Shi, S., and Yu, J. (2015), “Testing for Multiple Bubbles: Historical Episodes of Exuberance and Collapse in the S&P 500,” *International Economic Review*, 56, 1043–1078. Explosive-root diagnostics are relevant to bubble-state identification but are intentionally separate from the present conditional-severity engine.